DATA-260 HW10 – Evaluation, Judge Validation, Tracing and an Operations Dashboard

Pinal Pawar | SID4 7875 | PREFIX s7875 | Domain 3: Grocery supply and recall notices | Dashboard port 8676 (PORT_BASE 8675 + 1)

Stack: HW8 Kafka pipeline (s7875_inbox → Planner → s7875_tasks → Writer → s7875_drafts → Reviewer → s7875_final, dead letters on s7875_dead_letter) · HW9 Redis semantic cache (threshold 0.80 + same-notice filter) · MySQL s7875_rel (5,001 notices) · qwen3 via Ollama (Planner/Writer qwen3:1.7b, Reviewer qwen3:8b) · DeepEval 4.3.3 GEval · Streamlit 1.51 · MacBook Air M2, 16 GB, conda env data260.

Code: `hw10/` and `kafka_hw8/tracing.py` in github.com/PinalPawar/data260-7875 (tag `hw10`). Raw data: `reports/hw10/raw/`.

What was built

- **Traced ingress** (`hw10/ingress.py`): one uuid per request is the cache-lookup trace_id and is sent unchanged as the HW8 correlation_id. Cache hit → answer from Redis; miss → Kafka agents → evaluator.
- **Spans** (`kafka_hw8/tracing.py`): every stage appends one JSON line (trace_id, stage, start, end, duration, status, error) to `reports/hw10/raw/spans.jsonl`. The agents write their own spans from inside the HW8 agent loop.
- **Evaluator** (`hw10/evaluator.py`, `hw10/judge.py`): joins plan, draft and final by correlation_id and scores four criteria with GEval on a local judge, 1–5.
- **Judge validation** (`hw10/validate_judge.py`): 20 frozen drafts, my hand labels, 3 judge runs, Spearman / mean absolute difference / repeatability.
- **Latency experiment** (`hw10/latency_experiment.py`) and **Streamlit dashboard** (`hw10/dashboard.py`).

```
question --> cache_lookup (span) --hit--> answer from Redis  (no agents, no evaluation)
                                --miss-> s7875_inbox -> Planner -> s7875_tasks -> Writer ->
s7875_drafts
                                              -> Reviewer -> s7875_final -> evaluator (GEval) -> answer
         same uuid = trace_id = correlation_id on every message and every span
```

Setup and Environment

Kafka (s7875_kafka, 7 topics), Redis Stack (s7875-redis), MySQL s7875_rel and Ollama (qwen3:1.7b, qwen3:8b) run locally. Offline tests: 27 new HW10 checks (tracing, trace propagation through the agent loop, injected failure, evaluator bounds, statistics) and the 57 HW8 checks still pass with the new span hooks.

```
Downloading pytest_xdist-3.8.0-py3-none-any.whl (46 kB)
Downloading execnet-2.1.2-py3-none-any.whl (40 kB)
Installing collected packages: pytz, wcwidth, tzdata, tornado, toml, tabulate, smmap, pyfiglet, pyarrow, protobuf, portalocker, pluggy, packaging, opentelemetry-api, iniconfig, grpcio, execnet, click, cacheto
ols, blinker, backoff, rich, pytest, pydeck, prompt_toolkit, posthog, pandas, opentelemetry-semantic-conventions, gitdb, questionary, pytest-xdist, pytest-rerunfailures, pytest-repeat, pytest-asyncio, opentel
emetry-sdk, gitpython, deepeval, altair, streamlit
  Attempting uninstall: packaging
    Found existing installation: packaging 26.3
    Uninstalling packaging-26.3:
      Successfully uninstalled packaging-26.3
  Attempting uninstall: opentelemetry-api
    Found existing installation: opentelemetry-api 1.45.0
    Uninstalling opentelemetry-api-1.45.0:
      Successfully uninstalled opentelemetry-api-1.45.0
  Attempting uninstall: click
    Found existing installation: click 8.5.0
    Uninstalling click-8.5.0:
      Successfully uninstalled click-8.5.0
  Attempting uninstall: rich
    Found existing installation: rich 15.0.0
    Uninstalling rich-15.0.0:
      Successfully uninstalled rich-15.0.0
  Attempting uninstall: pandas
    Found existing installation: pandas 3.0.6
    Uninstalling pandas-3.0.6:
      Successfully uninstalled pandas-3.0.6
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.
huggingface-hub 1.32.0 requires click<9.0.0,>=8.4.2, but you have click 8.3.3 which is incompatible.
virtualenv 21.14.13 requires packaging>=26.3, but you have packaging 25.0 which is incompatible.
Successfully installed altair-5.5.0 backoff-2.2.1 blinker-1.9.0 cachetools-6.2.6 click-8.3.3 deepeval-4.3.3 execnet-2.1.2 gitdb-4.0.12 gitpython-3.2.1 grpcio-1.85.1 iniconfig-2.3.1 opentelemetry-api-1.45.2 op
entelemetry-sdk-1.45.2 opentelemetry-semantic-conventions-0.66b2 packaging-25.0 pandas-2.3.3 pluggy-1.7.0 portalocker-4.4.2 posthog-7.86.6 prompt_toolkit-3.0.53 protobuf-6.33.6 pyarrow-21.0.0 pydeck-0.9.3 pyf
iglet-1.0.4 pytest-9.2.0 pytest-asyncio-1.4.1.post1 pytest-repeat-0.9.4 pytest-rerunfailures-16.7 pytest-xdist-3.8.0 pytz-2026.6 questionary-2.1.1 rich-14.3.4 smmap-5.0.3 streamlit-1.51.0 tabulate-0.9.0 toml-
0.10.2 tornado-6.6.1 tzdata-2026.6 wcwidth-0.9.4
[+] up 4/4
 ✔ Container s7875_notice_worker Running                                0.0s
 ✔ Container s7875_notice_api    Running                                0.0s
 ✔ Container s7875_kafka         Healthy                                6.6s
 ✔ Container s7875_kafka_init    Exited                                 4.2s
===== Pinal Pawar | SID4 7875 | PREFIX s7875 | creating HW8 topics =====
+ kafka-topics --create --topic s7875_notice_requests --partitions 1 --replication-factor 1
+ kafka-topics --create --topic s7875_notice_responses --partitions 1 --replication-factor 1
+ kafka-topics --create --topic s7875_inbox --partitions 1 --replication-factor 1
+ kafka-topics --create --topic s7875_tasks --partitions 2 --replication-factor 1
+ kafka-topics --create --topic s7875_drafts --partitions 1 --replication-factor 1
+ kafka-topics --create --topic s7875_final --partitions 1 --replication-factor 1
+ kafka-topics --create --topic s7875_dead_letter --partitions 1 --replication-factor 1

+ kafka-topics --list
s7875_dead_letter
s7875_drafts
s7875_final
s7875_inbox
s7875_notice_requests
s7875_notice_responses
s7875_tasks

+ kafka-topics --describe (s7875_*)
Topic: s7875_notice_requests    TopicId: AGf98-OpQbWW5q2gzh0oOg PartitionCount: 1       ReplicationFactor: 1    Configs:
Topic: s7875_notice_responses   TopicId: ecp45EP0SPae6klvRJEndw PartitionCount: 1       ReplicationFactor: 1    Configs:
Topic: s7875_inbox      TopicId: wuI0sfqbRdaUEkybSWSVWA PartitionCount: 1      ReplicationFactor: 1     Configs:
Topic: s7875_tasks      TopicId: sTuOTLbRSSCKTl1eOnduxw PartitionCount: 2      ReplicationFactor: 1     Configs:
Topic: s7875_drafts     TopicId: 1FKQGHvfS6uqAMnrZUuSMQ PartitionCount: 1      ReplicationFactor: 1     Configs:
Topic: s7875_final      TopicId: oJFu_-Z_RB68aEk7_R0q2w PartitionCount: 1      ReplicationFactor: 1     Configs:
Topic: s7875_dead_letter        TopicId: 9psv2Z6HTU6aSFr0v9zgkQ PartitionCount: 1       ReplicationFactor: 1    Configs:
s7875-redis
(data260) Pinal Pawar | s7875 data260-7875 %
```

SS-1: HW10 requirements installed, Kafka up, all s7875_ topics created, Redis started

```
(data260) Pinal Pawar | s7875 data260-7875 % docker ps --format "table {{.Names}}\t{{.Status}}"
ollama list
NAMES                   STATUS
s7875_notice_api        Up 3 minutes
s7875_notice_worker     Up 3 minutes
s7875_kafka             Up 2 minutes (healthy)
s7875-redis             Up 2 minutes
s7875_mongo             Up 6 minutes
airflow-airflow-1       Up 6 minutes (healthy)
airflow-postgres-1      Up 6 minutes (healthy)
week13-airflow-1        Up About a minute (health: starting)
week13-mlflow-1         Up 6 minutes (healthy)
week13-postgres-1       Up 6 minutes (healthy)
NAME          ID              SIZE      MODIFIED
qwen3:8b      500a1f067a9f    5.2 GB    12 days ago
qwen3:1.7b    8f68893c685c    1.4 GB    2 months ago
(data260) Pinal Pawar | s7875 data260-7875 %
```

SS-2: Running containers and the two local Ollama models

```
(data260) Pinal Pawar | s7875 data260-7875 % python -c "import sys; sys.path.insert(0,'hw9'); from common import get_engine; from sqlalch
emy import text; print('MySQL OK, notices =', get_engine().connect().execute(text('select count(*) from notices')).scalar())"
MySQL OK, notices = 5001
(data260) Pinal Pawar | s7875 data260-7875 %
```

SS-3: MySQL s7875_rel reachable (5,001 recall notices)

```
(data260) Pinal Pawar | s7875 data260-7875 % python tests/test_hw10.py | tail -1
python tests/test_hw08_kafka.py | tail -1
tests/test_hw10.py: all 27 checks passed
tests/test_hw08_kafka.py: all 57 checks passed
(data260) Pinal Pawar | s7875 data260-7875 %
```

SS-4: Offline tests – HW10 27/27 and HW8 57/57

```
Last login: Tue Nov 10 09:37:33 on ttys001
(base) atharvamundada@Atharvas-MacBook-Air data260-7875 % cd ~/Desktop/data260-7875
conda activate data260
setopt PROMPT_SUBST
PROMPT='(${CONDA_DEFAULT_ENV}) Pinal Pawar | s7875 %1~ %# '
(data260) Pinal Pawar | s7875 data260-7875 % python planner.py
===== Pinal Pawar | SID4 7875 | PREFIX s7875 | PORT_BASE 8675 | planner (planner-88805) =====
2026-11-10T12:16:00.112-08:00 [PLANNER] in=s7875_inbox group=s7875_planners idempotency=True commit_mode=after max_retries=3 work_delay_s
=0.0 dead_letter=s7875_dead_letter
2026-11-10T12:16:00.151-08:00 [PLANNER] ASSIGNED partitions [0]
2026-11-10T12:16:00.156-08:00 [PLANNER] planner is ready and listening on s7875_inbox
```

SS-5: Planner listening on s7875_inbox

```
Last login: Tue Nov 10 12:15:16 on ttys002
(base) atharvamundada@Atharvas-MacBook-Air ~ % cd ~/Desktop/data260-7875
conda activate data260
setopt PROMPT_SUBST
PROMPT='(${CONDA_DEFAULT_ENV}) Pinal Pawar | s7875 %1~ %# '
(data260) Pinal Pawar | s7875 data260-7875 % python writer.py
===== Pinal Pawar | SID4 7875 | PREFIX s7875 | PORT_BASE 8675 | writer (writer-88811) =====
2026-11-10T12:16:13.100-08:00 [WRITER] in=s7875_tasks group=s7875_writers idempotency=True commit_mode=after max_retries=3 work_delay_s=0
.0 dead_letter=s7875_dead_letter
2026-11-10T12:16:13.122-08:00 [WRITER] ASSIGNED partitions [0, 1]
2026-11-10T12:16:13.124-08:00 [WRITER] writer is ready and listening on s7875_tasks
```

SS-6: Writer listening on s7875_tasks (2 partitions)

```
Last login: Tue Nov 10 12:15:17 on ttys003
(base) atharvamundada@Atharvas-MacBook-Air ~ % cd ~/Desktop/data260-7875
conda activate data260
setopt PROMPT_SUBST
PROMPT='(${CONDA_DEFAULT_ENV}) Pinal Pawar | s7875 %1~ %# '
(data260) Pinal Pawar | s7875 data260-7875 % HW8_MODEL=qwen3:8b python reviewer.py
===== Pinal Pawar | SID4 7875 | PREFIX s7875 | PORT_BASE 8675 | reviewer (reviewer-88817) =====
2026-11-10T12:16:26.206-08:00 [REVIEWER] in=s7875_drafts group=s7875_reviewers idempotency=True commit_mode=after max_retries=3 work_dela
y_s=0.0 dead_letter=s7875_dead_letter
2026-11-10T12:16:26.245-08:00 [REVIEWER] ASSIGNED partitions [0]
2026-11-10T12:16:26.249-08:00 [REVIEWER] reviewer is ready and listening on s7875_drafts
```

SS-7: Reviewer (qwen3:8b) listening on s7875_drafts

Part 1 – Streamlit Operations Dashboard

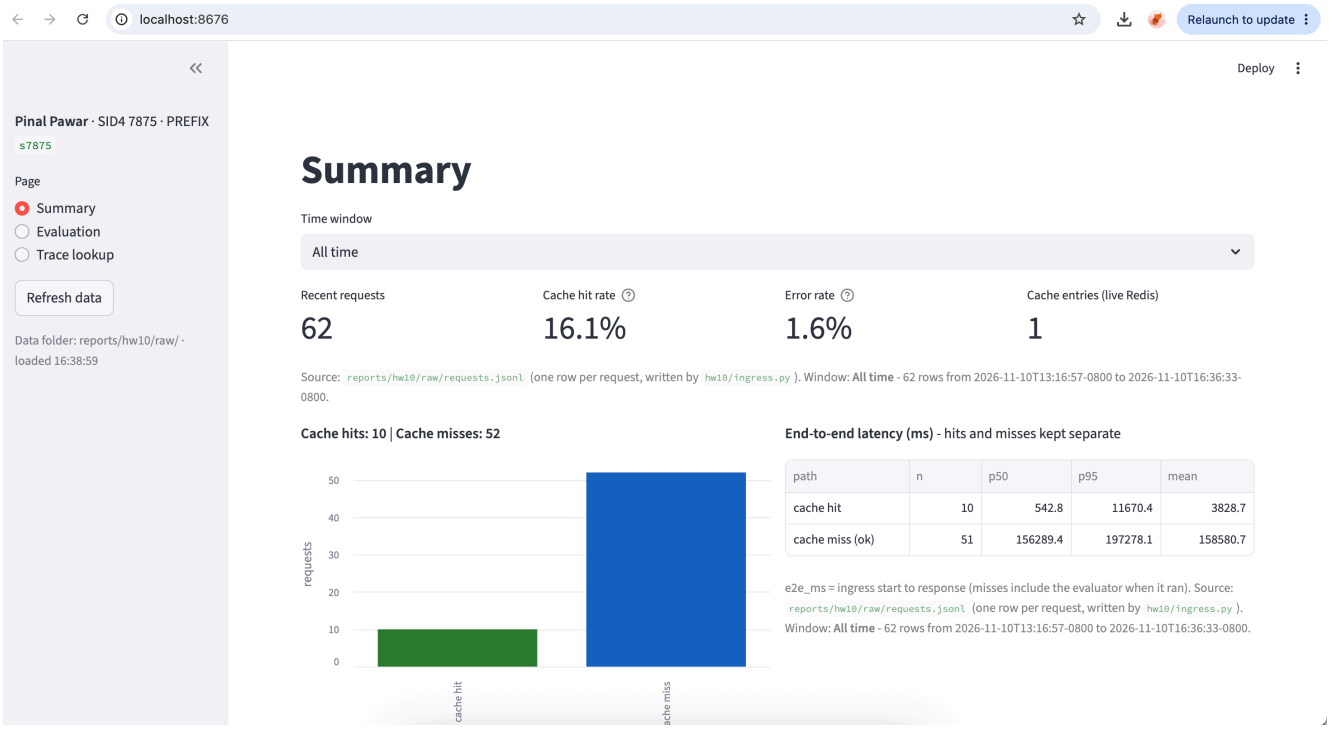
`streamlit run hw10/dashboard.py --server.port 8676`. Three pages in the sidebar; every number is read from my own logs in `reports/hw10/raw/` (no new data is generated by the dashboard). File loads use `@st.cache_data(ttl=10)` keyed on the file's modification time, so a rerun does not re-parse unchanged JSONL; the Redis connection used for the live cache-entry count is a `@st.cache_resource` (one client for the whole app).

```
@st.cache_data(ttl=10, show_spinner=False)
def load_file(path: str, mtime: float) -> list[dict]:    # re-parsed only when the file changes
    return load_jsonl(Path(path))

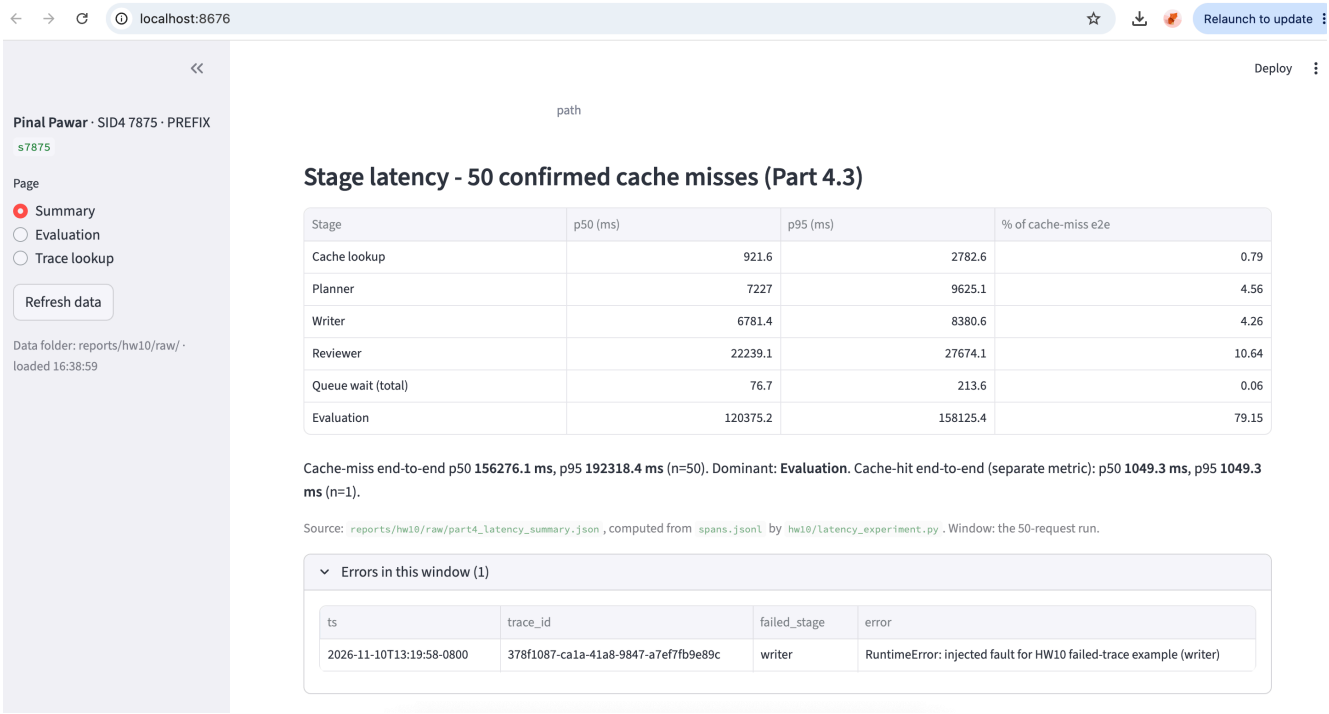
@st.cache_resource(show_spinner=False)
def redis_client():                                       # one connection reused by every rerun
    r = redis.Redis.from_url("redis://localhost:6379/0", decode_responses=True); r.ping(); return
r
```

1.1 Summary page

Request count, cache-hit rate (hits and misses shown as separate counts and bars), error rate (status ≠ ok: dead-lettered or timed out) and end-to-end latency p50/p95, with **cache hits and cache misses in separate rows**. A time-window selector (last 1 h / 24 h / 7 days / all) filters the requests; the caption under each block states the window, the row count, the first/last timestamp and the source file (`requests.jsonl`, one row per request written by `hw10/ingress.py`). The Part 4.3 stage table is shown underneath with its own source (`part4_latency_summary.json`).



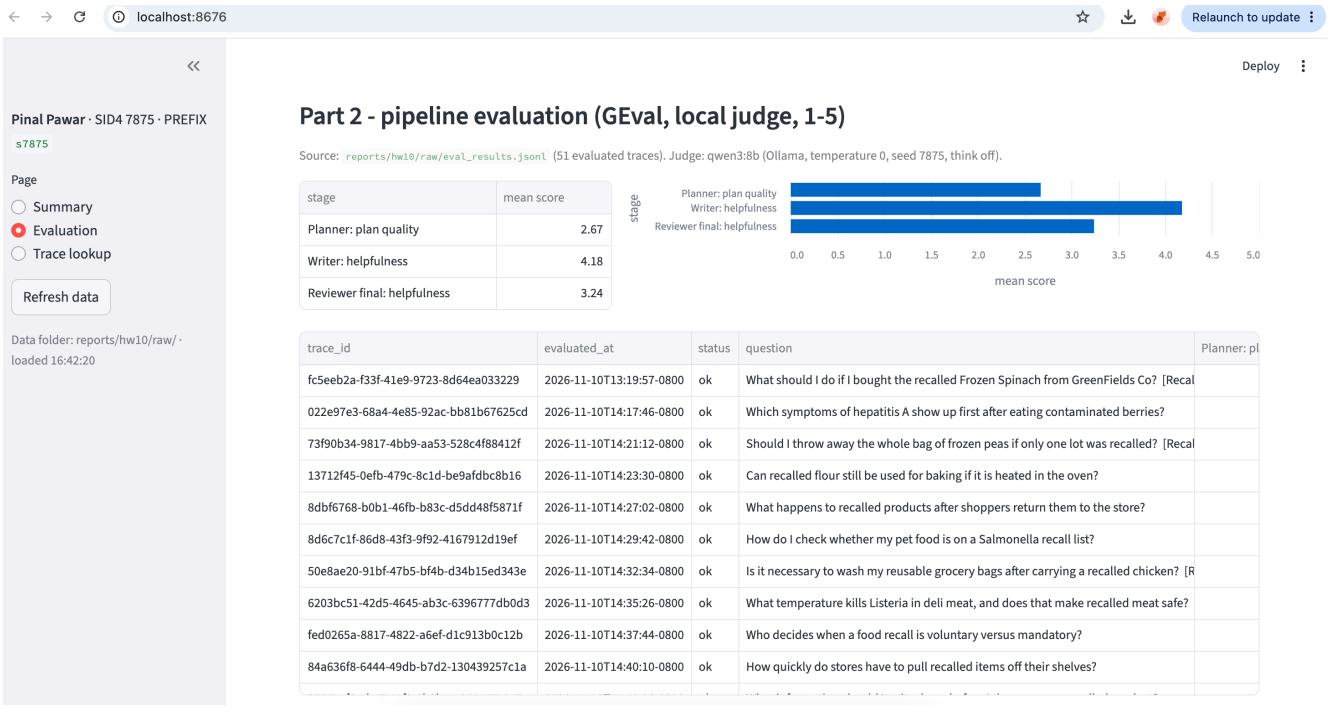
SS-8: Summary page – requests, hit rate, error rate, latency (hits vs misses), window and source



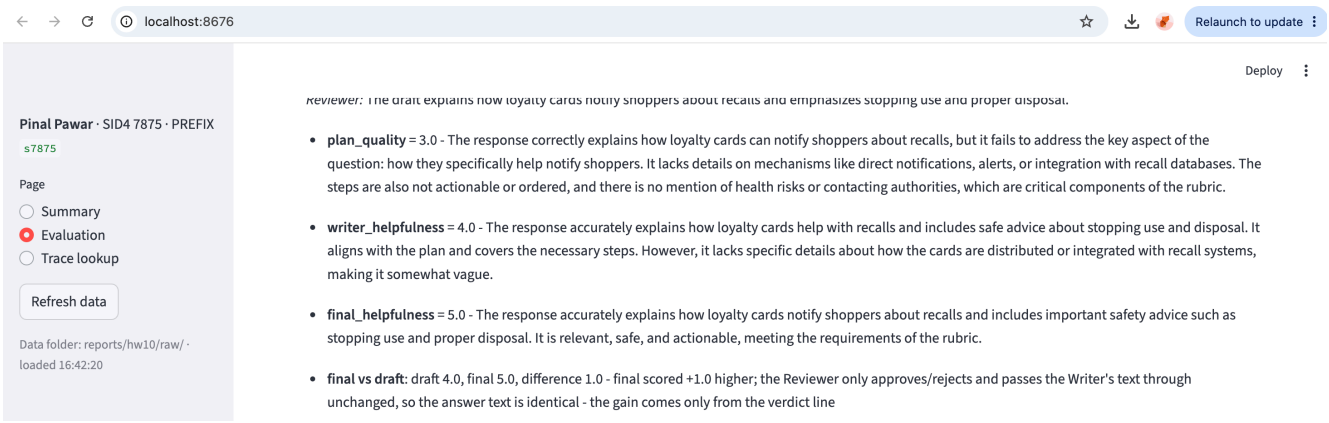
SS-9: Summary page – Part 4.3 stage-latency table and separate cache-hit latency

1.2 Evaluation page

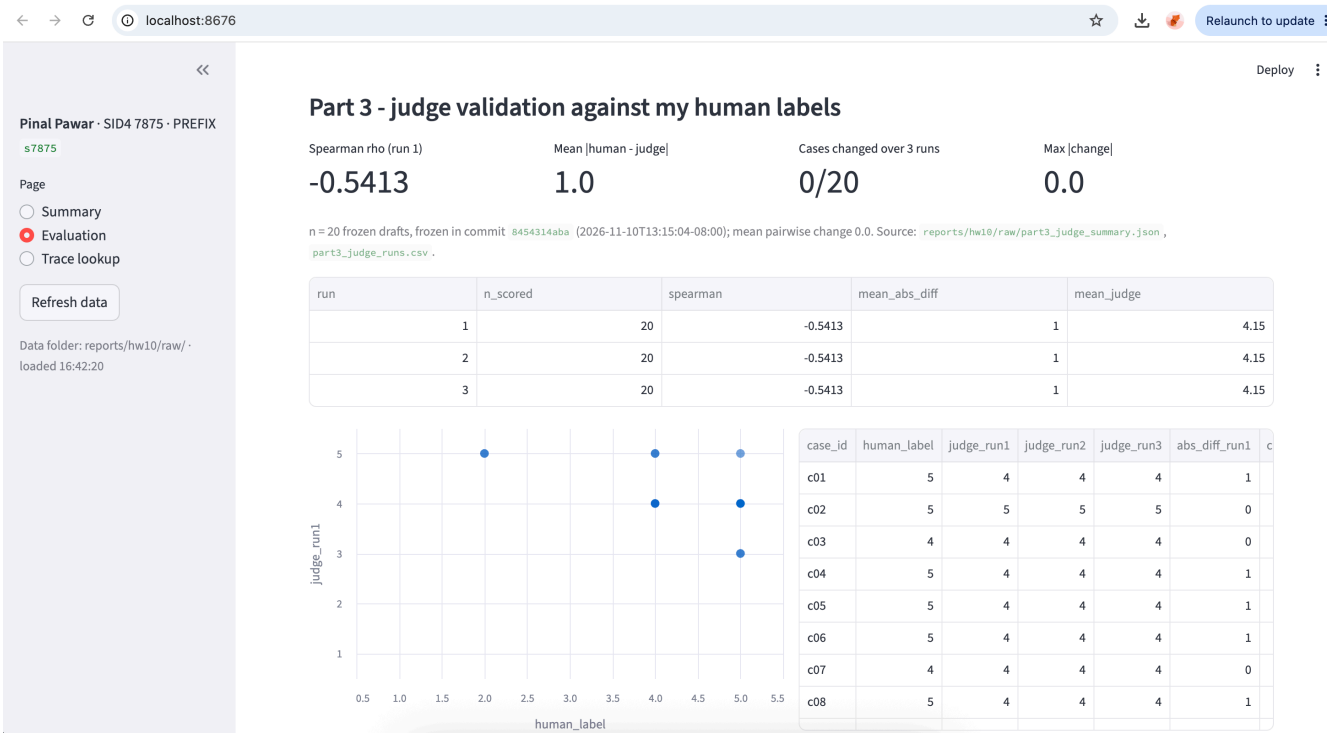
Part 2 scores by stage (mean per stage + per-trace table; selecting a trace shows the question, plan, draft, final answer, every score and the judge's reason). Part 3 shows Spearman, mean absolute difference, repeatability, the frozen commit, a human-vs-judge scatter and the raw per-case table. The interpretation text (`reports/hw10/interpretation.md`) is the same as in Part 2 of this report.



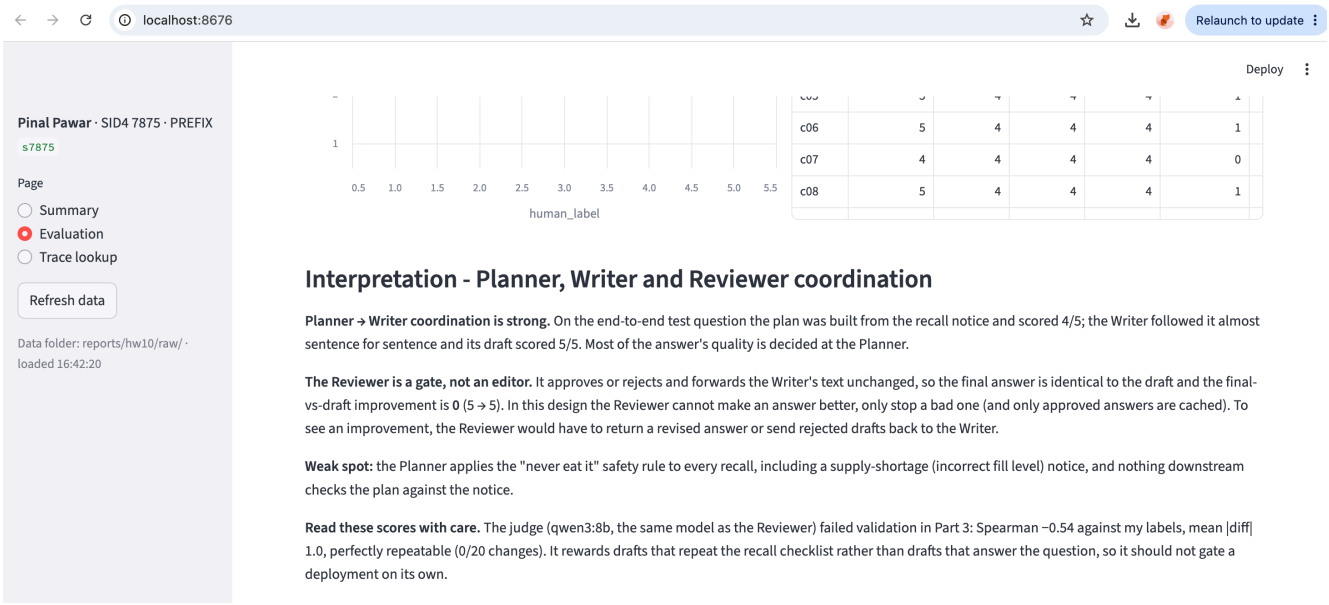
SS-10: Evaluation page – Part 2 scores by pipeline stage and case



SS-11: Evaluation page – one trace's details: scores, judge reasons and final-vs-draft



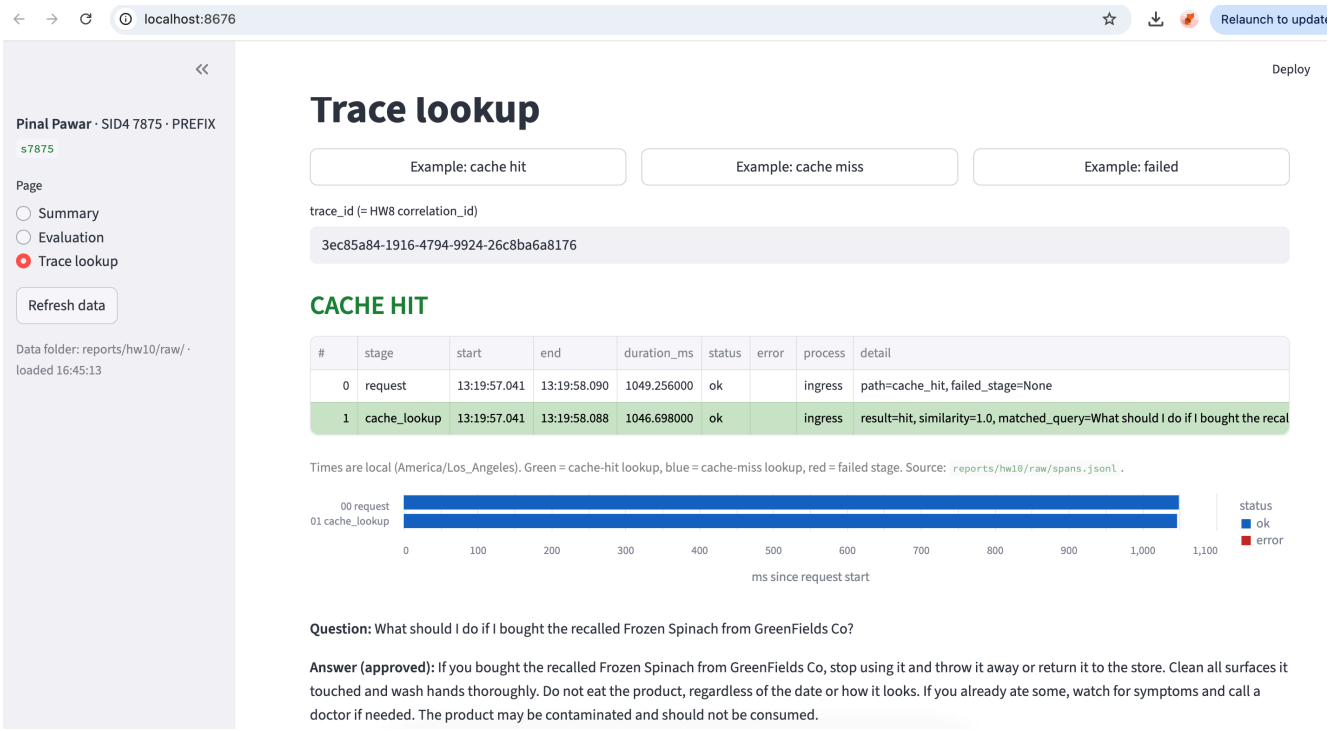
SS-12: Evaluation page – Part 3 judge validation (Spearman, MAD, repeatability, raw data)



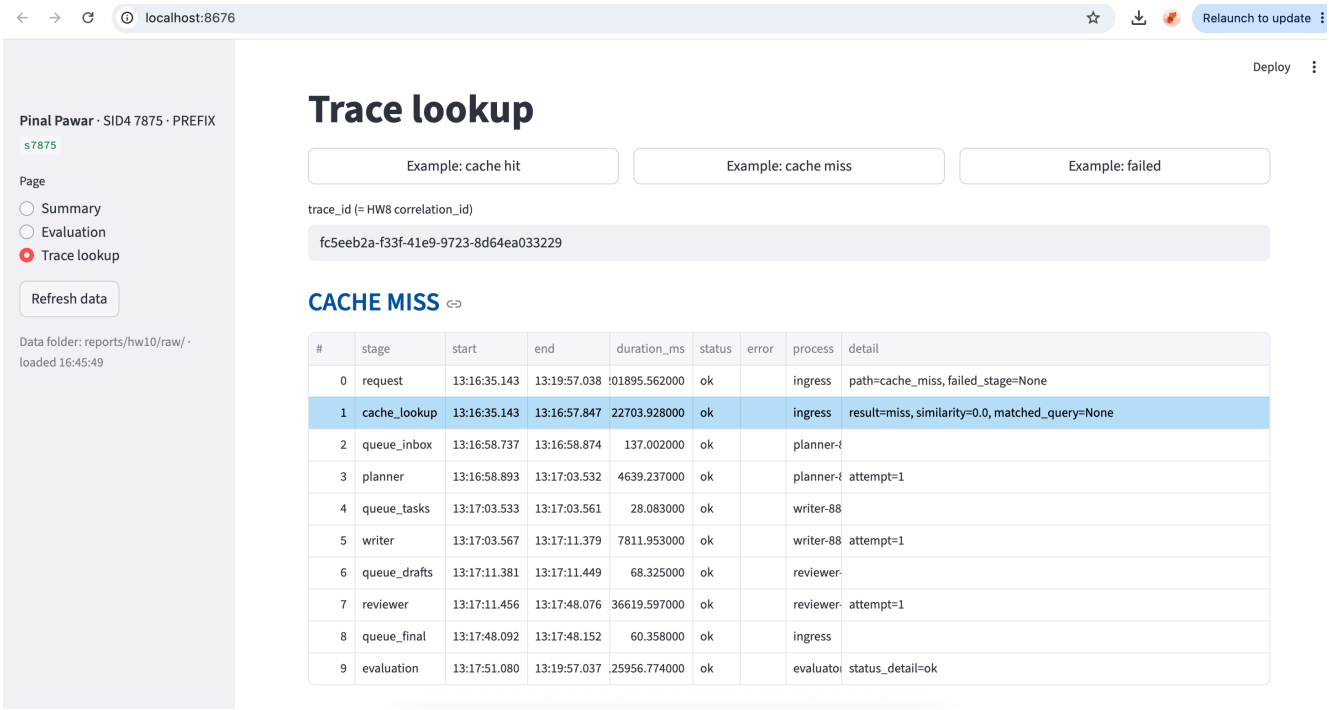
SS-13: Evaluation page – interpretation of Planner/Writer/Reviewer coordination

1.3 Trace lookup page

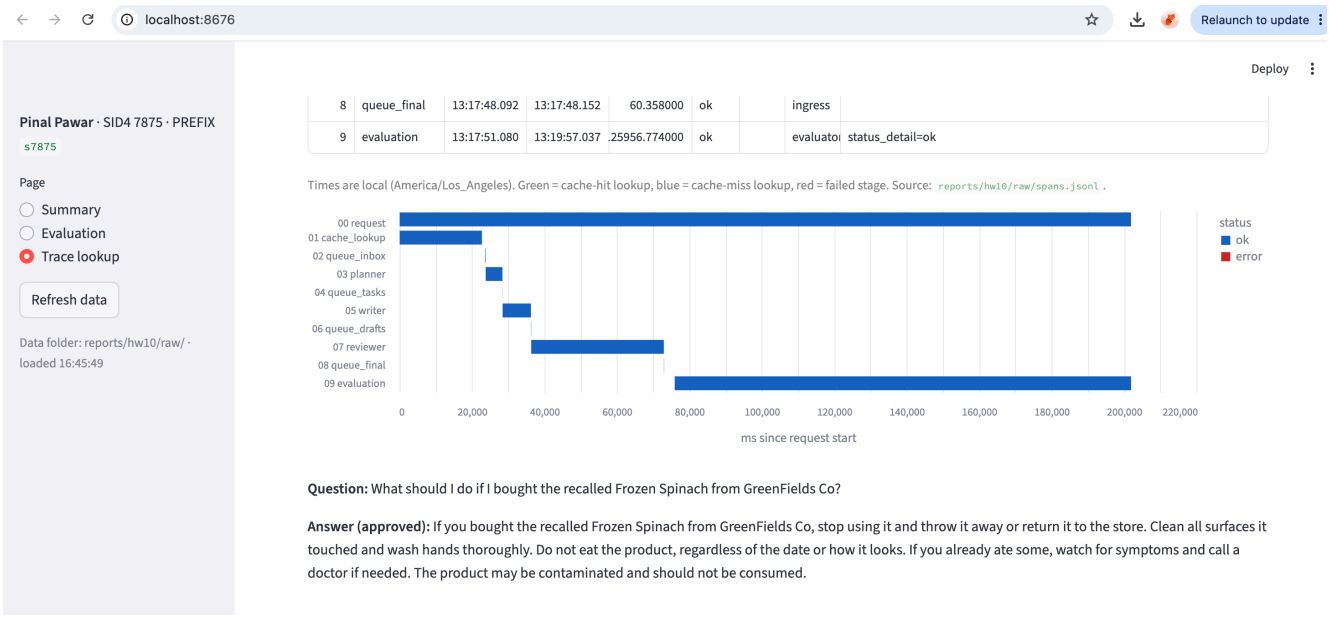
A `trace_id` input (plus one-click buttons for the three Part 4.2 examples) shows the journey in stage order with start, end, duration, status, error and process for each span, and a timeline chart. The header and row colours make the path obvious: green **CACHE HIT** (only cache_lookup + request), blue **CACHE MISS** (the full agent path), red **FAILED at <stage>** with the error on the failing rows.



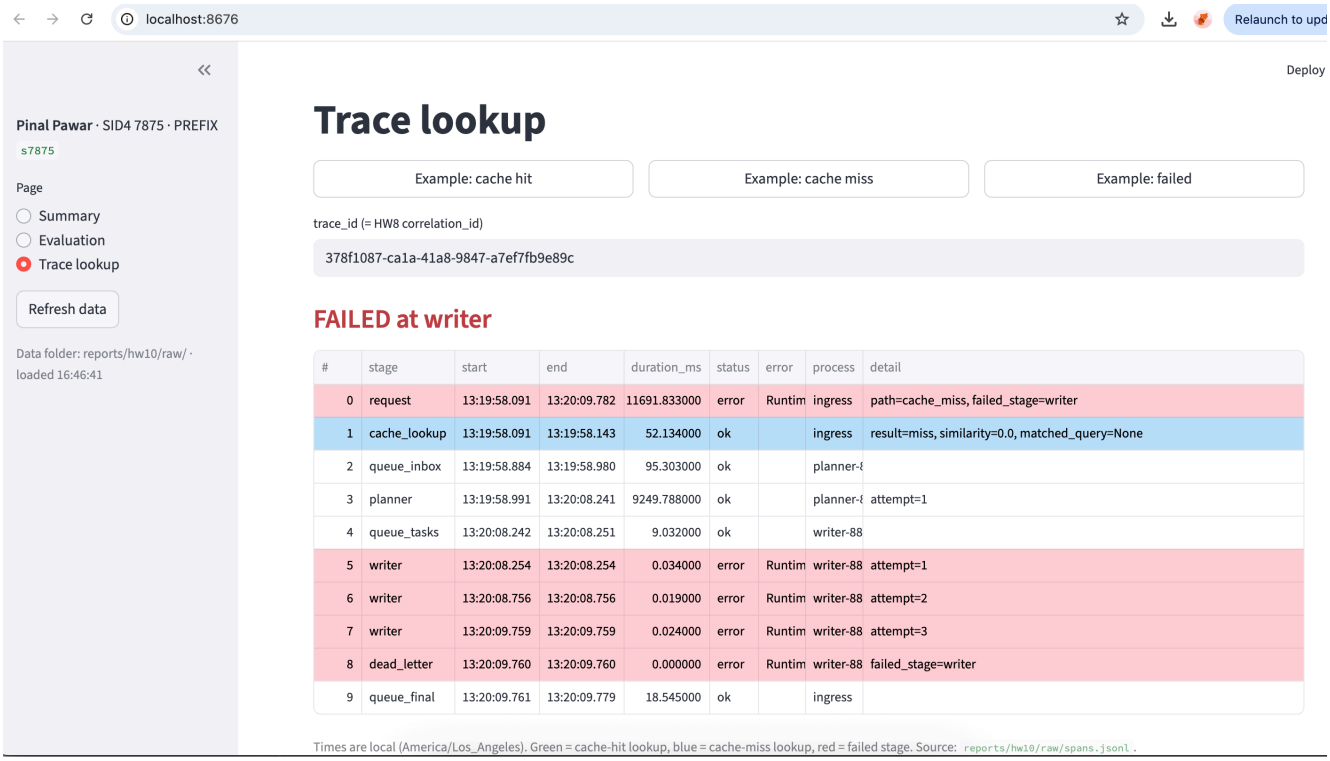
SS-14: Trace lookup – cache-hit trace (no agent stages)



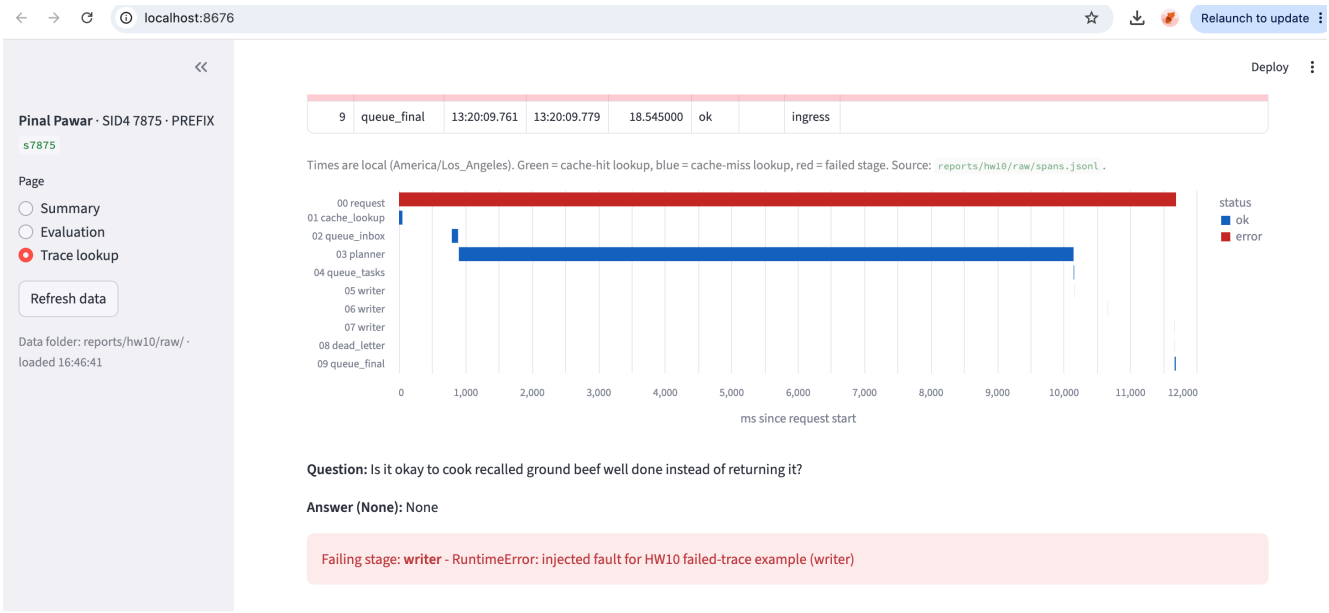
SS-15: Trace lookup – successful cache-miss trace through all agents and the evaluator



SS-16: Trace lookup – cache-miss timeline (evaluation is the longest bar)



SS-17: Trace lookup – failed trace: three Writer attempts and the dead letter



SS-18: Trace lookup – failed-trace timeline and the failing stage / error

Part 2 – Automated Evaluation of the Kafka Pipeline

2.1 Inputs, linkage and bounds

- **Join key:** after the Reviewer's message arrives on `s7875_final`, `evaluate_trace(trace_id)` reads `s7875_tasks` (plan), `s7875_drafts` (draft) and `s7875_final` (final) from the offsets recorded just before the question was sent, keeping only messages whose `correlation_id` equals the trace_id. The trace_id **is** the correlation_id, so no second identifier is joined later.
- **Reproducibility:** each result row in `eval_results.jsonl` stores the three original messages with topic, partition and offset, the extracted inputs, every score and reason, and the judge settings.
- **Judge:** DeepEval 4.3.3 `GEval` with a local model, `qwen3:8b` through Ollama, wrapped as a `DeepEvalBaseLLM` (no OpenAI key). Temperature 0, seed 7875, thinking off, max 400 tokens, JSON-schema-constrained output. Fixed `evaluation_steps` per criterion (no step-generation call) and a 5-band rubric with `score_range` (1,1)…(5,5), so the judge returns an integer **1 = very poor … 5 = excellent** — the same scale as the human labels.
- **Bounds:** the Ollama client has a 180 s timeout and each metric runs in a worker thread that is abandoned after 180 s (`judge_timeout`); connection or JSON errors return `judge_error`; each Kafka read has a 20 s timeout; a missing or malformed plan/draft/final returns status `incomplete` with the reason and makes no judge call. All three paths are covered by the offline tests.

```
def measure(metric, inp, out, timeout_s=180):           # hw10/judge.py (shortened)
    fut = pool.submit(metric.measure, LLMTestCase(input=inp, actual_output=out))
    try:
        fut.result(timeout=timeout_s)                    # hard bound per metric
        return {"status": "ok", "score_1_5": 1 + 4 * metric.score, "reason": metric.reason}
    except cf.TimeoutError:  return {"status": "judge_timeout", "score_1_5": None, ...}
    except Exception as e:   return {"status": "judge_error", "reason": str(e), ...}
```

2.2 Criteria and result on the end-to-end test question

Test question (cache miss, trace `fc5eeb2a-f33f-41e9-9723-8d64ea033229`): “What should I do if I bought the recalled Frozen Spinach from GreenFields Co?” The cache added the matching notice (RCL-2026-00001, Listeria in lot GF-2291). The Planner wrote 4 steps, the Writer a 69-word draft, and the Reviewer approved it.

Criterion	What is judged (input → output)	Score	Saved with it
Plan Quality	question → Planner plan	4	reason: safe, covers stop use, disposal, cleaning, medical advice
Writer Helpfulness	question + plan → Writer draft	5	trace_id; reason: accurate, safe, specific next steps
Final Answer Helpfulness	question → final answer + Reviewer verdict	5	the final answer text
Final-vs-Draft Improvement	same final-helpfulness metric on draft and on final	5 → 5, diff 0	both scores, difference, interpretation

```
(data260) Pinal Pawar | s7875 data260-7875 % python -c "import sys,json; sys.path.insert(0,'hw10'); import hw10cfg; from evaluator import
 print_row; print(hw10cfg.banner('Part 2 evaluation result')); print_row(json.loads(hw10cfg.EVAL_RESULTS.read_text().splitlines()[-1]))"
===== Pinal Pawar | SID4 7875 | PREFIX s7875 | HW10 Part 2 evaluation result =====
trace_id=fc5eeb2a-f33f-41e9-9723-8d64ea033229  status=ok  eval_ms=125955.0
  plan_quality         score=4.0  (ok) The response is safe and covers the necessary steps: stopping use, disposal, cleaning, and medical
 advice. How
  writer_helpfulness   score=5.0  (ok) The response is accurate, safe, and directly addresses the question with specific next steps, incl
uding stoppi
  final_helpfulness    score=5.0  (ok) The response is accurate, safe, and directly addresses the shopper's question by instructing them
to stop usin
  improvement          draft=5.0 final=5.0 diff=0.0  -> no improvement (difference 0); the Reviewer only approves/rejects and passes the
Writer's text through unchanged, so the answer text is identical
(data260) Pinal Pawar | s7875 data260-7875 %
```

SS-19: Part 2 evaluation of the end-to-end test question (scores and judge reasons)

All 51 evaluated requests (Part 2 evaluator run on every Part 4.3 request too)

Measure (n = 51, all status ok)	Result
Plan Quality, mean (min–max)	2.67 (1–5)
Writer Helpfulness, mean (min–max)	4.18 (2–5)
Final Answer Helpfulness, mean (min–max)	3.24 (1–5)
Final text identical to the Writer's draft	51 / 51
Reviewer verdicts	32 approved, 19 rejected (37%)
Final − draft (same metric), all	mean −0.67: 16 up, 15 equal, 20 down
… approved answers (n = 32)	mean +0.62: 16 up, 15 equal, 1 down
… rejected answers (n = 19)	mean −2.84: 19 of 19 down

```
(data260) Pinal Pawar | s7875 data260-7875 % python -c "
import json,statistics as s
R=[json.loads(l) for l in open('reports/hw10/raw/eval_results.jsonl') if l.strip()]
R=[r for r in R if r['status']=='ok']
g=lambda k:[r['scores'][k]['score_1_5'] for r in R]
d=[r['scores']['improvement']['difference'] for r in R]
print('n =',len(R))
for k in ('plan_quality','writer_helpfulness','final_helpfulness'): x=g(k); print(k,'mean',round(s.mean(x),2),'min',min(x),'max',max(x))
dr=[r['scores']['improvement']['draft_score'] for r in R]; print('draft (final metric) mean',round(s.mean(dr),2))
print('final-draft: mean',round(s.mean(d),2),' +',sum(x>0 for x in d),' 0',sum(x==0 for x in d),' -',sum(x<0 for x in d))
print('final text identical to draft:',sum(r['scores']['improvement']['final_text_identical_to_draft'] for r in R),'/',len(R))
print('reviewer verdicts:',{v:sum(r['inputs']['final_status']==v for r in R) for v in set(r['inputs']['final_status'] for r in R)})
"
n = 51
plan_quality mean 2.67 min 1.0 max 5.0
writer_helpfulness mean 4.18 min 2.0 max 5.0
final_helpfulness mean 3.24 min 1.0 max 5.0
draft (final metric) mean 3.9
final-draft: mean -0.67  + 16  0 15  - 20
final text identical to draft: 51 / 51
reviewer verdicts: {'approved': 32, 'rejected': 19}
(data260) Pinal Pawar | s7875 data260-7875 %
```

SS-20: Part 2 statistics over all 51 evaluated requests

```
(data260) Pinal Pawar | s7875 data260-7875 % python -c "
import json,statistics as s
R=[json.loads(l) for l in open('reports/hw10/raw/eval_results.jsonl') if l.strip()]
R=[r for r in R if r['status']=='ok']
for v in ('approved','rejected'):
    d=[r['scores']['improvement']['difference'] for r in R if r['inputs']['final_status']==v]
    print(v,'n',len(d),'mean diff',round(s.mean(d),2),' +',sum(x>0 for x in d),' 0',sum(x==0 for x in d),' -',sum(x<0 for x in d))
"
approved n 32 mean diff 0.62  + 16  0 15  - 1
rejected n 19 mean diff -2.84  + 0  0 0  - 19
(data260) Pinal Pawar | s7875 data260-7875 %
```

SS-21: Final − draft split by Reviewer verdict

What the scores say about coordination

- **Planner → Writer: strong.** The plan was built from the notice facts and the Writer followed it almost sentence for sentence (the draft repeats the four plan steps; Writer log in Part 4.2). That is why Writer Helpfulness equals the plan's content plus a closing line; most of the answer's quality is decided at the Planner.
- **The Reviewer's only effect is its verdict.** Over 51 requests the final text was identical to the draft every time, so every non-zero final − draft difference comes from the verdict line alone: “approved” nudges the judge up (+0.62), “rejected” pulls it down sharply (−2.84, all 19). The improvement metric therefore measures the judge's reaction to the verdict, not better answers.
- **The rejection loop is open.** The Reviewer rejected 37% of drafts, but nothing goes back to the Writer: the rejected draft is still the final message (it is only kept out of the cache). The coordination gap is the missing revise-and-resubmit step, not the Reviewer's judgement.
- **Plans score lowest (2.67).** The judge often calls the plans generic or not actionable; with the Writer averaging 4.18, the Writer recovers from weak plans, but these are judge scores and Part 3 shows how far to trust them.
- **Writer → Reviewer: a gate, not an editor.** The Reviewer (qwen3:8b) only approves or rejects; on approval it forwards the Writer's text unchanged. The final answer is byte-for-byte the draft, so the Final-vs-Draft difference is **0**. This is not a scoring accident: by design the Reviewer cannot improve an answer, it can only stop a bad one. Its value is the rejections (and only approved answers are cached), which this metric does not capture.
- **A coordination weakness visible in the traces:** for the ground-beef question used in the failed trace, the notice is a *supply shortage / incorrect fill level* recall, yet the Planner still wrote “the product must never be eaten … not safe” (Planner log in Part 4.2). The safety rule in the HW8 prompts is applied regardless of recall category, and nothing downstream checks the plan against the notice.
- **Caveat:** the judge (qwen3:8b) is the same model as the Reviewer, so the final-answer score may share the Reviewer's blind spots; Part 3 shows this judge should not be trusted on its own anyway.

Planned improvement if the final answer should actually get better: let the Reviewer return a revised answer (or send rejected drafts back to the Writer with the review), then re-run this metric; today a positive improvement score is impossible by construction.

Part 3 – Validating the Judge

3.1 Frozen evaluation cases

- `hw10/eval_cases.json`: 20 Writer drafts made with the HW8 Planner and Writer prompts and model (qwen3:1.7b, temperature 0, seed 7875). The 20 questions (`hw10/eval_questions.yaml`) were chosen to vary difficulty: plain and specific recall questions, multi-part ones, “can I still eat it” traps, a vague one (“recall?”) and an off-topic one (spinach-dip recipe).
- `hw10/human_labels.csv`: one helpfulness label 1–5 per draft, **typed by me** with `hw10/label_drafts.py` (it only displays the draft and reads my keystroke; no model is called).
- Both files were committed in **8454314** (2026-11-10 13:15) **before** the judge ever ran. `validate_judge.py` refuses to run unless both files are committed and unmodified, and records that commit (screenshot in 3.2). The labels were not changed after the judge results.

```
[c04]   7.9s  How do I find out whether the eggs in my fridge are part of
       draft: Check the packaging for a recall notice, look for the recall number and date. Contact the store where you purc
[c05]   8.3s  Is it safe to cook recalled ground beef to 160F instead of t
       draft: A recalled product must never be cooked, regardless of temperature. The recall indicates it is unsafe, and coo
[c06]  11.2s  My baby drank formula that was later recalled for botulism.
       draft: Stop using the recalled formula and throw it away. Clean surfaces it touched. Watch for symptoms of botulism,
[c07]  11.9s  How do I get a refund for a recalled product if I lost the r
       draft: Contact the store where the product was purchased to request a refund. Provide the item name, batch number, an
[c08]   8.0s  What is the difference between a Class I and a Class II reca
       draft: A Class I recall requires immediate action to prevent harm, such as stopping sales and removing the product fr
[c09]   8.7s  How should I clean my refrigerator after a recalled deli mea
       draft: Stop using the recalled deli meat and throw it away or return it to the store. Clean all surfaces it touched w
[c10]   6.9s  I'm pregnant and ate soft cheese that was recalled for Liste
       draft: Stop using the recalled soft cheese and throw it away. Clean surfaces it touched. Watch for illness symptoms a
[c11]  10.5s  Why do frozen berries get recalled for hepatitis A, and does
       draft: Frozen berries are recalled for hepatitis A due to the virus's ability to survive in frozen food. The virus ca
[c12]  10.8s  Where can I sign up to get recall alerts for the foods I buy
       draft: Check with your local health department or food safety authority for recall alerts. Subscribe to official food
[c13]   8.9s  My dog ate some recalled peanut butter. Is that a problem?
       draft: Your dog should not eat the recalled peanut butter and it must be discarded immediately. Clean the area where
[c14]   6.3s  A store is still selling a product I saw on a recall list. W
       draft: Stop using the product. Throw it away or return it to the store. Clean surfaces it touched. If you have alread
[c15]  10.2s  Can I donate unopened recalled canned goods to a food bank?
       draft: A recalled product must never be donated, regardless of its appearance or date. The food bank should be inform
[c16]  11.2s  What does 'undeclared allergen' mean on a recall notice, and
       draft: An "undeclared allergen" on a recall notice refers to a substance in the product that is not listed on the lab
[c17]   6.4s  recall?
       draft: If a product is recalled, stop using it immediately. Throw away or return the product as instructed. Clean all
[c18]  10.3s  I threw the package away. How can I tell if the chicken I at
       draft: Check the packaging for a recall notice or logo. Look for a date or batch number that matches the recall infor
[c19]   9.2s  What's a good recipe for spinach dip?
       draft: A simple spinach dip recipe: Wash and chop fresh spinach, mix with mayonnaise, lemon juice, garlic, and olive
[c20]   8.0s  How long after eating contaminated food do Listeria, Salmone
       draft: Symptoms of Listeria, Salmonella, and E. coli typically begin 2 to 4 weeks after eating contaminated food, tho

saved hw10/eval_cases.json (20 drafts)
(data260) Pinal Pawar | s7875 data260-7875 %
```

SS-22: 20 Writer drafts saved to eval_cases.json

```
  unsure, contact the store or manufacturer for confirmation. The recalled product must
  never be eaten, regardless of appearance or smell. If you already ate some, watch for
  illness symptoms and call a doctor if pregnant, older, or have a weak immune system.
Your helpfulness label 1-5: 2
Optional note (enter to skip):

================================================================================
c19  QUESTION: What's a good recipe for spinach dip?
PLAN:
  - Provide a simple recipe using fresh spinach, mayonnaise, lemon juice, and garlic.
  - Ensure the recipe is safe and follows food-safety rules, emphasizing not to eat recalled products.
  - Suggest cleaning surfaces and disposing of the dip if it is recalled.
  - Warn against eating any recalled product, regardless of appearance or smell.
DRAFT:
  A simple spinach dip recipe: Wash and chop fresh spinach, mix with mayonnaise, lemon
  juice, garlic, and olive oil. Serve cold. Do not use any recalled products. Clean
  surfaces touched, and dispose of the dip if it is recalled. If you ate any recalled
  product, watch for illness symptoms and seek medical attention. Never eat recalled
  food, regardless of appearance or smell.
Your helpfulness label 1-5: 4
Optional note (enter to skip):

================================================================================
c20  QUESTION: How long after eating contaminated food do Listeria, Salmonella and E. coli symptoms usually start?
PLAN:
  - Stop using the recalled product and throw it away or return it to the store. Clean surfaces it touched. If you already ate some, watc
h for symptoms and call a doctor if needed. Never suggest the product might still be safe.
DRAFT:
  Symptoms of Listeria, Salmonella, and E. coli typically begin 2 to 4 weeks after
  eating contaminated food, though they can appear as early as 1 day or as late as 10
  days. If you already ate some of the recalled product, watch for symptoms and seek
  medical attention if necessary. Never suggest the product might still be safe.
Your helpfulness label 1-5: 5
Optional note (enter to skip):

saved hw10/human_labels.csv - now commit both files
(data260) Pinal Pawar | s7875 data260-7875 %
```

SS-23: Hand-labelling the drafts (last two shown) and saving human_labels.csv

```
(data260) Pinal Pawar | s7875 data260-7875 % git add hw10/eval_cases.json hw10/human_labels.csv
git commit -m "HW10 Part 3.1: freeze 20 Writer drafts and human labels before judging"
git log --oneline -1 -- hw10/eval_cases.json hw10/human_labels.csv
[WARNING] Unstaged files detected.
[INFO] Stashing unstaged files to /Users/atharvamundada/.cache/pre-commit/patch1794345305-90198.
ruff check.....................................(no files to check)Skipped
[INFO] Restored changes from /Users/atharvamundada/.cache/pre-commit/patch1794345305-90198.
[main 8454314] HW10 Part 3.1: freeze 20 Writer drafts and human labels before judging
 2 files changed, 265 insertions(+)
 create mode 100644 hw10/eval_cases.json
 create mode 100644 hw10/human_labels.csv
8454314 (HEAD -> main) HW10 Part 3.1: freeze 20 Writer drafts and human labels before judging
(data260) Pinal Pawar | s7875 data260-7875 %
```

SS-24: Freeze commit 8454314 containing only the two files

3.2 Judge vs human labels

The judge is the Writer Helpfulness GEval metric from Part 2 (same steps, same 1–5 rubric, same settings). Per-case data (also in `reports/hw10/raw/part3_judge_runs.csv`, with the judge's reasons):

Case	Question	Human	Run 1	Run 2	Run 3	|H−J|
c01	What should I do if I bought the recalled Frozen Spinach from GreenFields Co?	5	4	4	4	1
c02	I already ate a salad with recalled romaine. What E. coli symptoms should I watch for?	5	5	5	5	0
c03	Can I still eat recalled cantaloupe if I wash the rind really well?	4	4	4	4	0
c04	How do I find out whether the eggs in my fridge are part of the Salmonella egg recall?	5	4	4	4	1
c05	Is it safe to cook recalled ground beef to 160F instead of throwing it away?	5	4	4	4	1
c06	My baby drank formula later recalled for botulism. What to watch for, who to call?	5	4	4	4	1
c07	How do I get a refund for a recalled product if I lost the receipt?	4	4	4	4	0
c08	What is the difference between a Class I and a Class II recall?	5	4	4	4	1
c09	How should I clean my refrigerator after a recalled deli meat was stored in it?	4	5	5	5	1
c10	I'm pregnant and ate soft cheese recalled for Listeria. What should I do?	4	4	4	4	0
c11	Why do frozen berries get recalled for hepatitis A, and does freezing kill it?	5	4	4	4	1
c12	Where can I sign up to get recall alerts for the foods I buy?	4	5	5	5	1
c13	My dog ate some recalled peanut butter. Is that a problem?	5	4	4	4	1
c14	A store is still selling a product I saw on a recall list. What should I do?	2	5	5	5	3
c15	Can I donate unopened recalled canned goods to a food bank?	5	3	3	3	2

Case	Question	Human	Run 1	Run 2	Run 3	|H−J|
c16	What does 'undeclared allergen' mean on a recall notice, and who is at risk?	5	4	4	4	1
c17	recall?	4	4	4	4	0
c18	I threw the package away. How can I tell if the chicken I ate was the recalled one?	2	5	5	5	3
c19	What's a good recipe for spinach dip?	4	4	4	4	0
c20	How long after eating contaminated food do Listeria, Salmonella and E. coli symptoms start?	5	3	3	3	2

Statistic (n = 20)	Value
Spearman rank correlation (human vs judge run 1; average ranks for ties)	−0.541
Mean absolute difference	1.00
Exact agreement / within 1 point	6 of 20 / 16 of 20
Mean score: human / judge	4.35 / 4.15
Score distribution: human	5 ×11, 4 ×7, 2 ×2 (no 1 or 3)
Score distribution: judge	5 ×5, 4 ×13, 3 ×2 (never below 3)
Spearman without c14 and c18	−0.349 (still negative)

```
(data260) Pinal Pawar | s7875 data260-7875 % python hw10/validate_judge.py
===== Pinal Pawar | SID4 7875 | PREFIX s7875 | HW10 Part 3 judge validation =====
frozen in commit 8454314aba at 2026-11-10T13:15:04-08:00
run 1 c01: score=4.0 (ok, 45903 ms)
run 1 c02: score=5.0 (ok, 20241 ms)
run 1 c03: score=4.0 (ok, 34819 ms)
run 1 c04: score=4.0 (ok, 28304 ms)
run 1 c05: score=4.0 (ok, 26352 ms)
run 1 c06: score=4.0 (ok, 27744 ms)
run 1 c07: score=4.0 (ok, 28720 ms)
run 1 c08: score=4.0 (ok, 25642 ms)
run 1 c09: score=5.0 (ok, 24172 ms)
run 1 c10: score=4.0 (ok, 22711 ms)
run 1 c11: score=4.0 (ok, 40483 ms)
run 1 c12: score=5.0 (ok, 28108 ms)
run 1 c13: score=4.0 (ok, 26609 ms)
run 1 c14: score=5.0 (ok, 25633 ms)
run 1 c15: score=3.0 (ok, 27370 ms)
run 1 c16: score=4.0 (ok, 31507 ms)
run 1 c17: score=4.0 (ok, 27842 ms)
run 1 c18: score=5.0 (ok, 21524 ms)
run 1 c19: score=4.0 (ok, 27298 ms)
run 1 c20: score=3.0 (ok, 29804 ms)
run 2 c01: score=4.0 (ok, 29392 ms)
run 2 c02: score=5.0 (ok, 25956 ms)
run 2 c03: score=4.0 (ok, 42434 ms)
run 2 c04: score=4.0 (ok, 34116 ms)
run 2 c05: score=4.0 (ok, 32736 ms)
run 2 c06: score=4.0 (ok, 37396 ms)
run 2 c07: score=4.0 (ok, 40941 ms)
run 2 c08: score=4.0 (ok, 37137 ms)
run 2 c09: score=5.0 (ok, 25816 ms)
run 2 c10: score=4.0 (ok, 29248 ms)
run 2 c11: score=4.0 (ok, 38286 ms)
run 2 c12: score=5.0 (ok, 26092 ms)
run 2 c13: score=4.0 (ok, 41592 ms)
run 2 c14: score=5.0 (ok, 31602 ms)
run 2 c15: score=3.0 (ok, 30031 ms)
run 2 c16: score=4.0 (ok, 31050 ms)
run 2 c17: score=4.0 (ok, 29209 ms)
run 2 c18: score=5.0 (ok, 23059 ms)
run 2 c19: score=4.0 (ok, 27178 ms)
run 2 c20: score=3.0 (ok, 29754 ms)
run 3 c01: score=4.0 (ok, 31531 ms)
run 3 c02: score=5.0 (ok, 20586 ms)
run 3 c03: score=4.0 (ok, 32315 ms)
run 3 c04: score=4.0 (ok, 29726 ms)
run 3 c05: score=4.0 (ok, 28032 ms)
run 3 c06: score=4.0 (ok, 35789 ms)
run 3 c07: score=4.0 (ok, 30245 ms)
run 3 c08: score=4.0 (ok, 32942 ms)
run 3 c09: score=5.0 (ok, 32755 ms)
run 3 c10: score=4.0 (ok, 22979 ms)
run 3 c11: score=4.0 (ok, 38523 ms)
run 3 c12: score=5.0 (ok, 25071 ms)
run 3 c13: score=4.0 (ok, 34024 ms)
run 3 c14: score=5.0 (ok, 26316 ms)
run 3 c15: score=3.0 (ok, 29044 ms)
run 3 c16: score=4.0 (ok, 31157 ms)
run 3 c17: score=4.0 (ok, 26176 ms)
run 3 c18: score=5.0 (ok, 21716 ms)
run 3 c19: score=4.0 (ok, 29358 ms)
run 3 c20: score=3.0 (ok, 32579 ms)
```

SS-25: validate_judge.py – frozen-commit check and the 60 judge calls

```
run 3 c01: score=4.0 (ok, 31531 ms)
run 3 c02: score=5.0 (ok, 20586 ms)
run 3 c03: score=4.0 (ok, 32315 ms)
run 3 c04: score=4.0 (ok, 29726 ms)
run 3 c05: score=4.0 (ok, 28032 ms)
run 3 c06: score=4.0 (ok, 35789 ms)
run 3 c07: score=4.0 (ok, 30245 ms)
run 3 c08: score=4.0 (ok, 32942 ms)
run 3 c09: score=5.0 (ok, 32755 ms)
run 3 c10: score=4.0 (ok, 22979 ms)
run 3 c11: score=4.0 (ok, 38523 ms)
run 3 c12: score=5.0 (ok, 25071 ms)
run 3 c13: score=4.0 (ok, 34024 ms)
run 3 c14: score=5.0 (ok, 26316 ms)
run 3 c15: score=3.0 (ok, 29044 ms)
run 3 c16: score=4.0 (ok, 31157 ms)
run 3 c17: score=4.0 (ok, 26176 ms)
run 3 c18: score=5.0 (ok, 21716 ms)
run 3 c19: score=4.0 (ok, 29358 ms)
run 3 c20: score=3.0 (ok, 32579 ms)

Spearman (human vs judge run 1) = -0.5413   mean |diff| = 1.0
Repeatability: 0/20 cases changed across 3 runs; mean pairwise |change| = 0.0, max = 0.0
saved reports/hw10/raw/part3_judge_runs.csv and reports/hw10/raw/part3_judge_summary.json
(data260) Pinal Pawar | s7875 data260-7875 %
```

SS-26: Spearman, mean |diff| and repeatability

```
(data260) Pinal Pawar | s7875 data260-7875 % cut -d, -f1-7 reports/hw10/raw/part3_judge_runs.csv
case_id,human_label,judge_run1,judge_run2,judge_run3,abs_diff_run1,changed_across_runs
c01,5,4.0,4.0,4.0,1.0,False
c02,5,5.0,5.0,5.0,0.0,False
c03,4,4.0,4.0,4.0,0.0,False
c04,5,4.0,4.0,4.0,1.0,False
c05,5,4.0,4.0,4.0,1.0,False
c06,5,4.0,4.0,4.0,1.0,False
c07,4,4.0,4.0,4.0,0.0,False
c08,5,4.0,4.0,4.0,1.0,False
c09,4,5.0,5.0,5.0,1.0,False
c10,4,4.0,4.0,4.0,0.0,False
c11,5,4.0,4.0,4.0,1.0,False
c12,4,5.0,5.0,5.0,1.0,False
c13,5,4.0,4.0,4.0,1.0,False
c14,2,5.0,5.0,5.0,3.0,False
c15,5,3.0,3.0,3.0,2.0,False
c16,5,4.0,4.0,4.0,1.0,False
c17,4,4.0,4.0,4.0,0.0,False
c18,2,5.0,5.0,5.0,3.0,False
c19,4,4.0,4.0,4.0,0.0,False
c20,5,3.0,3.0,3.0,2.0,False
(data260) Pinal Pawar | s7875 data260-7875 %
```

SS-27: Raw per-case data (part3_judge_runs.csv)

Why the judge disagrees with me

The four largest disagreements, with the judge's own reasons (screenshot below):

Case	Human	Judge	Judge's reason (shortened)	My reading
c14 store still selling	2	5	“accurate and safe … stopping use, discarding, returning, cleaning … excellent”	Never answers the question (report it to the store / FDA); just the generic recall checklist.
c18 threw package away	2	5	“accurate, safe … aligns with the plan … clear and actionable”	Never says how to identify the chicken (receipt, store purchase history, recall notice details).
c15 donate canned goods	5	3	“fails to address … lacks specific next steps for the donor”	Directly answers “no, never donate”; the judge wanted more checklist.
c20 symptom onset	5	3	“fails to address the recall plan … stopping use, discarding …”	Penalised for missing checklist; but the draft is also wrong for Salmonella/ E. coli (hours–days, not 2–4 weeks), which I missed.

```
(data260) Pinal Pawar | s7875 data260-7875 % python -c "import csv; [print(r['case_id'], 'human', r['human_label'], 'judge', r['judge_run
1'], '|', r['reason_run1'], '\n') for r in csv.DictReader(open('reports/hw10/raw/part3_judge_runs.csv')) if r['case_id'] in ('c14','c18',
'c15','c20')]"
c14 human 2 judge 5.0 | The response is accurate and safe, providing clear next steps such as stopping use, discarding, returning, cleani
ng, and seeking medical attention. It also adds a helpful reminder about not eating the product regardless of appearance or date, which r
einforces the recall advice. The response is specific, complete, and easy to act on, meeting all the criteria for an excellent response.

c15 human 5 judge 3.0 | The response correctly states that recalled food must never be donated and provides clear guidance on disposal an
d informing the food bank. However, it fails to address the specific question about whether unopened recalled canned goods can be donated
, instead focusing on general advice. It also lacks specific next steps for the donor, such as checking the recall details or contacting
the manufacturer, which are critical for a complete answer.

c18 human 2 judge 5.0 | The response is accurate, safe, and addresses the question with specific next steps. It correctly advises against
 eating the recalled product, mentions consulting a doctor for high-risk individuals, and aligns with the plan provided in the input. The
 advice is clear and actionable.

c20 human 5 judge 3.0 | The response provides accurate information about the onset of symptoms for Listeria, Salmonella, and E. coli, whi
ch aligns with the question. However, it fails to address the recall plan from the Input, such as stopping use of the product, discarding
 it, returning it, cleaning surfaces, and seeking medical attention if needed. This omission makes the response incomplete and less usefu
l for a shopper following a food recall.

(data260) Pinal Pawar | s7875 data260-7875 %
```

SS-28: Judge reasons for c14, c15, c18, c20

- **The judge measures checklist compliance, not helpfulness.** It scores a draft high when it repeats the recall checklist (stop, discard/return, clean, see a doctor) and low when it omits it, regardless of whether the question was answered. I scored whether the shopper's actual question

was answered. Part of this bias is my rubric: the safety sentence I put in every evaluation step pushes the judge towards the checklist.
- **Compressed scale.** The judge never used 1 or 2 and gave 4 to 13 of 20 drafts, so it barely separates drafts. My labels are also top-heavy (18 of 20 are 4 or 5), which limits any rank correlation; heavy ties on both sides make Spearman unstable at n = 20.
- **Human labels are a reference, not ground truth.** On c20 the judge's lower score was right for a different reason (a factual error I did not notice). The labels stay frozen; this is reported, not corrected.

3.3 Repeatability

Three runs over the same 20 drafts with identical inputs and settings (qwen3:8b, temperature 0, seed 7875, thinking off, max 400 tokens, fixed evaluation steps, 1–5 rubric):

Measure	Value
Cases whose score changed across the 3 runs	0 of 20 (0%)
Mean pairwise absolute change	0.0
Maximum absolute change	0
Judge failures / timeouts	0 of 60 calls (20–46 s per call)

The judge is perfectly repeatable at temperature 0 with a fixed seed. That shows the setup is deterministic, not that the judge is right: it repeats the same wrong ranking three times.

Would I trust this judge to gate a deployment?

No. A judge whose ranking is negatively correlated with a human reference ($\rho = -0.54$) would approve changes that make answers less helpful and block changes that make them better. Its only strength is consistency. Before using any judge as a gate I would require:

1. **Agreement:** Spearman ≥ 0.7 and mean absolute difference ≤ 0.5 on a fresh frozen set of at least 50–100 drafts that covers the whole 1–5 range, including deliberately bad drafts (off-topic, factually wrong, unsafe).
2. **A second human labeller** and their inter-rater agreement, as the ceiling the judge is compared to.
3. **A revised rubric** that scores “answers the question asked” and factual accuracy separately from safety, then re-validated on new frozen cases (not the same 20, to avoid tuning to them).
4. **A judge from a different model family** than the Reviewer (e.g., llama3.1:8b), to remove self-preference, plus a check that known-bad drafts are scored ≤ 2.
5. **Stability evidence beyond temperature 0:** repeat runs after an Ollama/model update and with paraphrased inputs; use the judge only as a regression alarm with human review, never as the sole gate.

Part 4 – Distributed Tracing and Latency

4.1 Trace propagation

- **One identifier.** The ingress makes one uuid, uses it as the trace_id of the `cache_lookup` span, and sends it as both `correlation_id` and `trace_id` on `s7875_inbox`. The HW8 agent loop carries it onto every outgoing message (`out.value.setdefault("trace_id", …)`), so the same value appears on s7875_tasks, s7875_drafts, s7875_final, s7875_dead_letter and in the evaluator.
- **Spans** (one JSON line each in `spans.jsonl`): `request` (root, end to end), `cache_lookup` (with result = hit/miss, similarity, threshold), `queue_inbox`, `planner`, `queue_tasks`, `writer`, `queue_drafts`, `reviewer`, `queue_final`, `evaluation`, `dead_letter`. Fields: trace_id, stage,

start, end (local ISO time), duration_ms, status, error, process and attributes (topic, partition, offset, attempt).

- **Queue wait** is measured, not guessed: every sender stamps `sent_at_ms` on the message and the consumer closes a `queue_*` span at receive time. Retried attempts each write their own span, so a failure shows every attempt.

```
# kafka_hw8/agent_base.py (HW10 additions, shortened)
trace_id = _trace_queue_wait(self, msg, record, corr)     # queue_<topic> span: sent_at_ms -> now
t_start = now_ms(); _maybe_inject_fault(self, msg)
outputs = await self.handle(msg, record)
write_span(trace_id, self.name, t_start, now_ms(), "ok", None, self.agent_id,
attempt=attempt, ...)
...                                                        # on exception: same span with status
"error"
for out in outputs:
    out.value.setdefault("trace_id", trace_id); out.value["sent_at_ms"] = now_ms()
```

4.2 Trace examples

`python hw10/ingress.py --examples` produced all three (IDs saved in `reports/hw10/raw/trace_examples.json` and loaded by the dashboard's example buttons):

Trace	trace_id	Path / outcome	End to end
Cache miss (success)	fc5eeb2a-f33f-41e9-9723-8d64ea033229	lookup miss → Planner 4.6 s → Writer 7.8 s → Reviewer 36.6 s (approved) → evaluation 126 s	201.9 s
Cache hit	3ec85a84-1916-4794-9924-26c8ba6a8176	same question; lookup hit, similarity 1.000; no agents, no evaluation	1.05 s
Failed	378f1087-ca1a-41a8-9847-a7ef7fb9e89c	Planner ok → Writer failed 3× (RuntimeError) → s7875_dead_letter	11.7 s

Failed trace – failing stage and error: stage `writer`, error `RuntimeError: injected fault for HW10 failed-trace example (writer)`. The failure is deliberate: the ingress sets `inject_fault="writer"` on the message and the Writer raises before calling the model. The normal HW8 failure path then runs unchanged: retries after 0.5 s and 1 s backoff (attempts 1/3, 2/3, 3/3), then the original message goes to `s7875_dead_letter` with `failed_agent=writer`, and the offset is committed. Each attempt and the dead letter are separate error spans, and the ingress records the request as status error, failed_stage writer.

```
(data260) Pinal Pawar | s7875 data260-7875 % python hw10/ingress.py --examples
===== Pinal Pawar | SID4 7875 | PREFIX s7875 | HW10 Part 4 traced ingress =====
cleared 0 entries from s7875:hw10cache

--- cache-miss request (evaluated): What should I do if I bought the recalled Frozen Spinach from GreenFields Co?
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and f
  trace_id   = fc5eeb2a-f33f-41e9-9723-8d64ea033229
  path       = cache_miss  is_cached=False  similarity=0.0  status=ok  e2e_ms=201895.6
  reviewer   = approved  The draft correctly instructs to stop using, throw away, or return the product, clean surfaces, and not eat it, while advising to se
  answer     = If you bought the recalled Frozen Spinach from GreenFields Co, stop using it and throw it away or return it to the store. Clean all surfaces t
 produc

--- same question again (expect cache hit): What should I do if I bought the recalled Frozen Spinach from GreenFields Co?
  trace_id   = 3ec85a84-1916-4794-9924-26c8ba6a8176
  path       = cache_hit  is_cached=True  similarity=1.0  status=ok  e2e_ms=1049.3
  reviewer   = approved
  answer     = If you bought the recalled Frozen Spinach from GreenFields Co, stop using it and throw it away or return it to the store. Clean all surfaces t
 produc

--- failed request (inject_fault=writer): Is it okay to cook recalled ground beef well done instead of returning it?
  trace_id   = 378f1087-ca1a-41a8-9847-a7ef7fb9e89c
  path       = cache_miss  is_cached=False  similarity=0.0  status=error  e2e_ms=11691.8
  FAILED at  = writer  error=RuntimeError: injected fault for HW10 failed-trace example (writer)

saved reports/hw10/raw/trace_examples.json:
{
  "cache_miss": "fc5eeb2a-f33f-41e9-9723-8d64ea033229",
  "cache_hit": "3ec85a84-1916-4794-9924-26c8ba6a8176",
  "failed": "378f1087-ca1a-41a8-9847-a7ef7fb9e89c",
  "failed_stage": "writer",
  "failed_error": "RuntimeError: injected fault for HW10 failed-trace example (writer)"
}
(data260) Pinal Pawar | s7875 data260-7875 %
```

SS-29: The three example requests and their trace IDs

```
2026-11-10T13:16:58.879-08:00 [PLANNER] RECEIVE         corr=fc5eeb2a-f33f-41e9-9723-8d64ea033229 topic=s7875_inbox p=0 off=54 key=fc5eeb2a-f33f-41e9-9723-8d64ea033229 {"correlation_id": "fc5eeb2a-f33f-41e9-9
723-8d64ea033229", "trace_id": "fc5eeb2a-f33f-41e9-9723-8d64ea033229", "stage": "question", "question": "What should I do if I bought the recalled Frozen Spinach from GreenFields Co?\n\n[Recall notice on file
 - use these facts: product: Frozen Spinach (10oz); manufacturer: GreenFields Co; notice code: RCL-2026-00001; category: bacterialContamination; units affected: 2919; details: Possible Listeria contamination
in lot GF-2291. Updated scope to 10oz bags only.]", "asked_at": "2026-11-10T13:16:58.737-08:00", "trace": [], "source": "hw10_ingress", "sent_at_ms": 1794345418737.489}
2026-11-10T13:16:58.893-08:00 [PLANNER] PROCESS-START   corr=fc5eeb2a-f33f-41e9-9723-8d64ea033229 attempt=1 work_delay_s=0.0
2026-11-10T13:17:03.532-08:00 [PLANNER] PROCESS-DONE    corr=fc5eeb2a-f33f-41e9-9723-8d64ea033229 steps=4 model=qwen3:1.7b via Ollama
2026-11-10T13:17:03.533-08:00 [PLANNER] SEND            corr=fc5eeb2a-f33f-41e9-9723-8d64ea033229 topic=s7875_tasks {"correlation_id": "fc5eeb2a-f33f-41e9-9723-8d64ea033229", "stage": "plan", "question": "Wha
t should I do if I bought the recalled Frozen Spinach from GreenFields Co?\n\n[Recall notice on file - use these facts: product: Frozen Spinach (10oz); manufacturer: GreenFields Co; notice code: RCL-2026-0000
1; category: bacterialContamination; units affected: 2919; details: Possible Listeria contamination in lot GF-2291. Updated scope to 10oz bags only.]", "plan": ["Stop using the Frozen Spinach and throw it awa
y or return it to the store.", "Clean all surfaces it touched and wash hands thoroughly.", "Do not eat the product, regardless of the date or how it looks.", "If you already ate some, watch for symptoms and c
all a doctor if needed."], "trace": [{"agent": "planner", "agent_id": "planner-88805", "read_from": "s7875_inbox", "partition": 0, "offset": 54, "at": "2026-11-10T13:17:03.532-08:00"}], "trace_id": "fc5eeb2a-
f33f-41e9-9723-8d64ea033229", "sent_at_ms": 1794345423533.844}
2026-11-10T13:17:03.691-08:00 [PLANNER] COMMIT          corr=fc5eeb2a-f33f-41e9-9723-8d64ea033229 p=0 next_offset=55
2026-11-10T13:19:58.982-08:00 [PLANNER] RECEIVE         corr=378f1087-ca1a-41a8-9847-a7ef7fb9e89c topic=s7875_inbox p=0 off=55 key=378f1087-ca1a-41a8-9847-a7ef7fb9e89c {"correlation_id": "378f1087-ca1a-41a8-9
847-a7ef7fb9e89c", "trace_id": "378f1087-ca1a-41a8-9847-a7ef7fb9e89c", "stage": "question", "question": "Is it okay to cook recalled ground beef well done instead of returning it?\n\n[Recall notice on file -
use these facts: product: Ground Beef (batch 4999); manufacturer: ACME Foods; notice code: RCL-2026-04999; category: supplyShortage; units affected: 2081; details: Distributor reported a packaging shortage ca
using incorrect fill levels; recall issued for lot #L14999.]", "asked_at": "2026-11-10T13:19:58.884-08:00", "trace": [], "source": "hw10_ingress", "sent_at_ms": 1794345598884.947, "inject_fault": "writer"}
2026-11-10T13:19:58.991-08:00 [PLANNER] PROCESS-START   corr=378f1087-ca1a-41a8-9847-a7ef7fb9e89c attempt=1 work_delay_s=0.0
2026-11-10T13:20:08.241-08:00 [PLANNER] PROCESS-DONE    corr=378f1087-ca1a-41a8-9847-a7ef7fb9e89c steps=4 model=qwen3:1.7b via Ollama
2026-11-10T13:20:08.242-08:00 [PLANNER] SEND            corr=378f1087-ca1a-41a8-9847-a7ef7fb9e89c topic=s7875_tasks {"correlation_id": "378f1087-ca1a-41a8-9847-a7ef7fb9e89c", "stage": "plan", "question": "Is
it okay to cook recalled ground beef well done instead of returning it?\n\n[Recall notice on file - use these facts: product: Ground Beef (batch 4999); manufacturer: ACME Foods; notice code: RCL-2026-04999; c
ategory: supplyShortage; units affected: 2081; details: Distributor reported a packaging shortage causing incorrect fill levels; recall issued for lot #L14999.]", "plan": ["The recall notice explicitly states
 that the product must never be eaten, regardless of its appearance, smell, or texture.", "Cooking the product well done does not make it safe; it is still recalled and potentially harmful.", "The user should
 immediately stop using the product and throw it away or return it to the store.", "Clean any surfaces the product touched to prevent contamination or spread of any possible illness."], "trace": [{"agent": "p
lanner", "agent_id": "planner-88805", "read_from": "s7875_inbox", "partition": 0, "offset": 55, "at": "2026-11-10T13:20:08.241-08:00"}], "trace_id": "378f1087-ca1a-41a8-9847-a7ef7fb9e89c", "sent_at_ms": 17943
45608242.301, "inject_fault": "writer"}
2026-11-10T13:20:08.263-08:00 [PLANNER] COMMIT          corr=378f1087-ca1a-41a8-9847-a7ef7fb9e89c p=0 next_offset=56
```

SS-30: Planner handling both agent-path traces; the correlation_id equals the trace_id

```
2026-11-10T13:20:08.254-08:00 [WRITER] PROCESS-START    corr=378f1087-ca1a-41a8-9847-a7ef7fb9e89c attempt=1 work_delay_s=0.0
2026-11-10T13:20:08.255-08:00 [WRITER] RETRY            corr=378f1087-ca1a-41a8-9847-a7ef7fb9e89c attempt=1/3 error=RuntimeError: injected fault for HW10 failed-trace example (writer)
2026-11-10T13:20:08.756-08:00 [WRITER] PROCESS-START    corr=378f1087-ca1a-41a8-9847-a7ef7fb9e89c attempt=2 work_delay_s=0.0
2026-11-10T13:20:08.757-08:00 [WRITER] RETRY            corr=378f1087-ca1a-41a8-9847-a7ef7fb9e89c attempt=2/3 error=RuntimeError: injected fault for HW10 failed-trace example (writer)
2026-11-10T13:20:09.758-08:00 [WRITER] PROCESS-START    corr=378f1087-ca1a-41a8-9847-a7ef7fb9e89c attempt=3 work_delay_s=0.0
2026-11-10T13:20:09.760-08:00 [WRITER] RETRY            corr=378f1087-ca1a-41a8-9847-a7ef7fb9e89c attempt=3/3 error=RuntimeError: injected fault for HW10 failed-trace example (writer)
2026-11-10T13:20:09.761-08:00 [WRITER] DEAD-LETTER      corr=378f1087-ca1a-41a8-9847-a7ef7fb9e89c topic=s7875_dead_letter attempts=3 error=RuntimeError: injected fault for HW10 failed-trace example (writer)
2026-11-10T13:20:09.761-08:00 [WRITER] SEND             corr=378f1087-ca1a-41a8-9847-a7ef7fb9e89c topic=s7875_dead_letter {"correlation_id": "378f1087-ca1a-41a8-9847-a7ef7fb9e89c", "failed_agent": "writer", "a
gent_id": "writer-88811", "error": "RuntimeError: injected fault for HW10 failed-trace example (writer)", "attempts": 3, "source": {"topic": "s7875_tasks", "partition": 1, "offset": 27}, "original": {"correla
tion_id": "378f1087-ca1a-41a8-9847-a7ef7fb9e89c", "stage": "plan", "question": "Is it okay to cook recalled ground beef well done instead of returning it?\n\n[Recall notice on file - use these facts: product:
 Ground Beef (batch 4999); manufacturer: ACME Foods; notice code: RCL-2026-04999; category: supplyShortage; units affected: 2081; details: Distributor reported a packaging shortage causing incorrect fill leve
ls; recall issued for lot #L14999.]", "plan": ["The recall notice explicitly states that the product must never be eaten, regardless of its appearance, smell, or texture.", "Cooking the product well done does
 not make it safe; it is still recalled and potentially harmful.", "The user should immediately stop using the product and throw it away or return it to the store.", "Clean any surfaces the product touched to
 prevent contamination or spread of any possible illness."], "trace": [{"agent": "planner", "agent_id": "planner-88805", "read_from": "s7875_inbox", "partition": 0, "offset": 55, "at": "2026-11-10T13:20:08.24
1-08:00"}], "trace_id": "378f1087-ca1a-41a8-9847-a7ef7fb9e89c", "sent_at_ms": 1794345608242.301, "inject_fault": "writer"}, "dead_lettered_at": "2026-11-10T13:20:09.760-08:00", "trace_id": "378f1087-ca1a-41a8
-9847-a7ef7fb9e89c", "sent_at_ms": 1794345609761.2102, "inject_fault": "writer"}
2026-11-10T13:20:09.786-08:00 [WRITER] COMMIT           corr=378f1087-ca1a-41a8-9847-a7ef7fb9e89c p=1 next_offset=28
```

SS-31: Writer – failed trace: RETRY 1/3, 2/3, 3/3 (0.5 s and 1 s backoff), DEAD-LETTER, offset committed

```
2026-11-10T13:17:11.454-08:00 [REVIEWER] RECEIVE        corr=fc5eeb2a-f33f-41e9-9723-8d64ea033229 topic=s7875_drafts p=0 off=54 key=fc5eeb2a-f33f-41e9-9723-8d64ea033229 {"correlation_id": "fc5eeb2a-f33f-41e9
-9723-8d64ea033229", "question": "What should I do if I bought the recalled Frozen Spinach from GreenFields Co?\n\n[Recall notice on file - use these facts: product: Frozen Spinach (10oz); manufacturer: Green
Fields Co; notice code: RCL-2026-00001; category: bacterialContamination; units affected: 2919; details: Possible Listeria contamination in lot GF-2291. Updated scope to 10oz bags only.]", "plan": ["Stop usin
g the Frozen Spinach and throw it away or return it to the store.", "Clean all surfaces it touched and wash hands thoroughly.", "Do not eat the product, regardless of the date or how it looks.", "If you alrea
dy ate some, watch for symptoms and call a doctor if needed."], "stage": "draft", "draft": "If you bought the recalled Frozen Spinach from GreenFields Co, stop using it and throw it away or return it to the s
tore. Clean all surfaces it touched and wash hands thoroughly. Do not eat the product, regardless of the date or how it looks. If you already ate some, watch for symptoms and call a doctor if needed. The prod
uct may be contaminated and should not be consumed.", "trace": [{"agent": "planner", "agent_id": "planner-88805", "read_from": "s7875_inbox", "partition": 0, "offset": 54, "at": "2026-11-10T13:17:03.532-08:00
"}, {"agent": "writer", "agent_id": "writer-88811", "read_from": "s7875_tasks", "partition": 1, "offset": 26, "at": "2026-11-10T13:17:11.379-08:00"}], "trace_id": "fc5eeb2a-f33f-41e9-9723-8d64ea033229", "sent
_at_ms": 1794345431381.137}
2026-11-10T13:17:11.456-08:00 [REVIEWER] PROCESS-START   corr=fc5eeb2a-f33f-41e9-9723-8d64ea033229 attempt=1 work_delay_s=0.0
2026-11-10T13:17:48.070-08:00 [REVIEWER] PROCESS-DONE    corr=fc5eeb2a-f33f-41e9-9723-8d64ea033229 status=approved model=qwen3:8b via Ollama
2026-11-10T13:17:48.092-08:00 [REVIEWER] SEND            corr=fc5eeb2a-f33f-41e9-9723-8d64ea033229 topic=s7875_final {"correlation_id": "fc5eeb2a-f33f-41e9-9723-8d64ea033229", "status": "approved", "answer":
"If you bought the recalled Frozen Spinach from GreenFields Co, stop using it and throw it away or return it to the store. Clean all surfaces it touched and wash hands thoroughly. Do not eat the product, rega
rdless of the date or how it looks. If you already ate some, watch for symptoms and call a doctor if needed. The product may be contaminated and should not be consumed.", "review": "The draft correctly instru
cts to stop using, throw away, or return the product, clean surfaces, and not eat it, while advising to seek medical help if consumed.", "question": "What should I do if I bought the recalled Frozen Spinach f
rom GreenFields Co?\n\n[Recall notice on file - use these facts: product: Frozen Spinach (10oz); manufacturer: GreenFields Co; notice code: RCL-2026-00001; category: bacterialContamination; units affected: 29
19; details: Possible Listeria contamination in lot GF-2291. Updated scope to 10oz bags only.]", "plan": ["Stop using the Frozen Spinach and throw it away or return it to the store.", "Clean all surfaces it t
ouched and wash hands thoroughly.", "Do not eat the product, regardless of the date or how it looks.", "If you already ate some, watch for symptoms and call a doctor if needed."], "model": "qwen3:8b via Ollam
a", "trace": [{"agent": "planner", "agent_id": "planner-88805", "read_from": "s7875_inbox", "partition": 0, "offset": 54, "at": "2026-11-10T13:17:03.532-08:00"}, {"agent": "writer", "agent_id": "writer-88811"
, "read_from": "s7875_tasks", "partition": 1, "offset": 26, "at": "2026-11-10T13:17:11.379-08:00"}, {"agent": "reviewer", "agent_id": "reviewer-88817", "read_from": "s7875_drafts", "partition": 0, "offset": 5
4, "at": "2026-11-10T13:17:48.076-08:00"}], "finished_at": "2026-11-10T13:17:48.076-08:00", "trace_id": "fc5eeb2a-f33f-41e9-9723-8d64ea033229", "sent_at_ms": 1794345468092.52}
2026-11-10T13:17:48.258-08:00 [REVIEWER] COMMIT          corr=fc5eeb2a-f33f-41e9-9723-8d64ea033229 p=0 next_offset=55
```

SS-32: Reviewer approves the cache-miss draft and sends it to s7875_final

4.3 Stage-latency experiment (50 confirmed cache misses)

- **Questions:** 50 new food-recall questions (`hw10/latency_questions.yaml`, 55 available as spares) that were never asked before.
- **Confirmed misses:** each request runs through the traced ingress with `store=False`, so nothing from the experiment enters the cache and one question cannot make a later one hit. A request is counted only if its `cache_lookup` span records result = miss and the request finished ok; any hit would be excluded and replaced by the next spare question.
- **Full path:** every request goes cache lookup → inbox → Planner → tasks → Writer → drafts → Reviewer → final → evaluator (four GEval metrics), exactly as in Part 2. Percentiles use linear interpolation; "% of end to end" = the stage's summed time ÷ the summed end-to-end time of the 50 requests. Queue wait (total) = queue_inbox + queue_tasks + queue_drafts + queue_final.

Stage	p50 (ms)	p95 (ms)	% of cache-miss end to end
Cache lookup	921.6	2,782.6	0.79%
Planner	7,227	9,625.1	4.56%
Writer	6,781.4	8,380.6	4.26%
Reviewer	22,239.1	27,674.1	10.64%
Queue wait (total)	76.7	213.6	0.06%
Evaluation	120,375.2	158,125.4	79.15%
End to end (cache miss)	156,276.1	192,318.4	100%

n = 50 confirmed misses (all cache_lookup spans = miss: true, excluded: 0). Unattributed time (embedding inside the ingress, Kafka polling between spans): 0.54%. When the run finished only one cache hit existed (1,049 ms); the table below adds the later cold and warm hits.

Cache-hit latency, kept separate (same question, all 10 hits with similarity 1.000; the dashboard Summary shows p50 543 ms / p95 11,670 ms over all 10):

Cache-hit request	n	End to end
Warm hit (same process already loaded the embedding model)	6	32–36 ms (first of these, which loaded the model: 5.5 s)
Hit right after a miss in --examples (model already loaded)	1	1.05 s
Cold one-shot CLI hit (new process: model load + HF Hub check)	3	8.7–14.0 s

```
(data260) Pinal Pawar | s7875 data260-7875 % python -c "
import json,sys; sys.path.insert(0,'hw10'); import hw10cfg
print(hw10cfg.banner('Part 4 cache-hit latency: cold vs warm (from requests.jsonl)'))
for l in open('reports/hw10/raw/requests.jsonl'):
    r=json.loads(l)
    if r['is_cached']: print(f\"  {r['kind']:<12} {r['ts']}  trace_id={r['trace_id']}  e2e_ms={r['e2e_ms']}\")
"
===== Pinal Pawar | SID4 7875 | PREFIX s7875 | HW10 Part 4 cache-hit latency: cold vs warm (from requests.jsonl) =====
  example_hit  2026-11-10T13:19:58-0800  trace_id=3ec85a84-1916-4794-9924-26c8ba6a8176  e2e_ms=1049.3
  demo         2026-11-10T16:34:41-0800  trace_id=b7bb1e77-d07e-46b2-8ec4-4f0123db772d  e2e_ms=13965.6
  demo         2026-11-10T16:34:51-0800  trace_id=d7ec64a0-13c8-460b-ac7b-39b8674f940e  e2e_ms=8865.1
  demo         2026-11-10T16:35:00-0800  trace_id=7bd75d40-bc36-4f37-ab2a-f09067dd4806  e2e_ms=8701.2
  warm_hit     2026-11-10T16:36:33-0800  trace_id=8754cb28-8eb2-4f04-a0a2-2083d1ee97c5  e2e_ms=5532.8
  warm_hit     2026-11-10T16:36:33-0800  trace_id=add4de7b-fb59-4d56-bcf1-5b5ba794a75a  e2e_ms=36.3
  warm_hit     2026-11-10T16:36:33-0800  trace_id=f8881369-a9f5-4dd0-8b53-ac6d69a066c2  e2e_ms=34.6
  warm_hit     2026-11-10T16:36:33-0800  trace_id=97d08f5d-ffea-4a59-9aa6-4458498d5275  e2e_ms=32.4
  warm_hit     2026-11-10T16:36:33-0800  trace_id=df9bd833-069e-47f5-8fd8-529ac287bd1d  e2e_ms=33.4
  warm_hit     2026-11-10T16:36:33-0800  trace_id=5fe6ee01-0f96-4f80-b0b0-f5da1c69407a  e2e_ms=36.1
(data260) Pinal Pawar | s7875 data260-7875 %
```

SS-33: All 10 cache-hit requests from requests.jsonl: cold vs warm

```
(data260) Pinal Pawar | s7875 data260-7875 % HF_HUB_OFFLINE=1 caffeinate -i python hw10/latency_experiment.py
===== Pinal Pawar | SID4 7875 | PREFIX s7875 | HW10 Part 4.3 stage-latency experiment =====
[ 1/50] MISS e2e= 148160.8 ms  status=ok  Which symptoms of hepatitis A show up first after eatin
[ 2/50] MISS e2e= 205514.9 ms  status=ok  Should I throw away the whole bag of frozen peas if onl
[ 3/50] MISS e2e= 137978.4 ms  status=ok  Can recalled flour still be used for baking if it is he
```

SS-34: latency_experiment.py running – every request a confirmed MISS

```
[44/50] MISS e2e= 155163.9 ms  status=ok  Why do recalls sometimes expand to more products weeks
[45/50] MISS e2e= 185552.7 ms  status=ok  Can heavy metals in baby food be removed by cooking?
[46/50] MISS e2e= 159271.3 ms  status=ok  How do I explain a food recall to elderly parents who d
[47/50] MISS e2e= 176746.1 ms  status=ok  Should I report symptoms to my local health department
[48/50] MISS e2e= 191900.3 ms  status=ok  Is microwaving leftovers enough to kill bacteria from a
[49/50] MISS e2e= 178353.2 ms  status=ok  What should campers do if their packed food is recalled
[50/50] MISS e2e= 183869.7 ms  status=ok  How do grocery loyalty cards help notify shoppers about

Stage                    p50 (ms)    p95 (ms)  % of e2e
Cache lookup                921.6      2782.6     0.79%
Planner                    7227.0      9625.1     4.56%
Writer                     6781.4      8380.6     4.26%
Reviewer                  22239.1     27674.1    10.64%
Queue wait (total)           76.7       213.6     0.06%
Evaluation               120375.2    158125.4    79.15%
End-to-end (miss)        156276.1    192318.4      100%
unattributed (embedding/ingress overhead): 0.54%
dominant contributor: Evaluation   n=50   all confirmed misses: True
cache-hit e2e (separate): {'n': 1, 'p50_ms': 1049.3, 'p95_ms': 1049.3}
saved reports/hw10/raw/part4_latency_summary.json
(data260) Pinal Pawar | s7875 data260-7875 %
```

SS-35: Stage-latency table printed at the end of the 50-request run

Dominant contributor and what I would optimise first

- **Dominant contributor: Evaluation, 79% of cache-miss end-to-end time** (p50 120 s, p95 158 s). It is four GEval calls on qwen3:8b, run one after another, inline on the request path. Without it the agent path is about 37 s at p50 (lookup 0.9 + Planner 7.2 + Writer 6.8 + Reviewer 22.2 + queues 0.08).
- **First optimisation: take evaluation off the request path.** Make the evaluator its own Kafka consumer group on `s7875_final` that scores asynchronously (or a sample of traffic, e.g. 10%), so the shopper gets the answer as soon as the Reviewer approves it. That alone removes ~79% of the end-to-end time; inside the evaluator, the four metrics can run in parallel and could use a smaller judge (after re-validating it, Part 3).
- **Next: the Reviewer**, which is ~60% of the remaining agent path (22 s p50 on qwen3:8b vs ~7 s each for the 1.7b Planner and Writer). Options: run the cheap HW8 rule check first and call the 8b model only when it passes, keep the model loaded (`OLLAMA_KEEP_ALIVE`), and cap its output length.
- **Kafka is not the bottleneck:** total queue wait is 77 ms p50 / 214 ms p95 (0.06%). Cache lookup is 0.9 s p50 / 2.8 s p95 during the experiment, but a warm cache hit on an idle Mac takes only ~34

ms end to end (table below): the lookup was slowed by sharing the M2 with the Ollama models that had just finished the previous request's evaluation.

- **Cache-hit latency (separate metric) depends on warm vs cold process.** One-shot CLI requests pay for loading the embedding model (and a Hugging Face version check) on every call; a long-running service pays it once. So the ingress should run as a long-lived service with the model preloaded.
- All 50 requests had `cache_lookup` result = miss (0 excluded) and finished ok. The Mac was moved during the run with the lid open and no network; everything is local, so the run was not interrupted.

Reproducibility and Verification

```
pip install -r requirements-hw10.txt
docker compose -f docker-compose.kafka.yml up -d && bash scripts/hw08_create_topics.sh && docker
start s7875-redis
python planner.py | python writer.py | HW8_MODEL=qwen3:8b python reviewer.py      # three
terminals
python hw10/build_eval_cases.py && python hw10/label_drafts.py && git commit ...   # Part 3.1
freeze first
python hw10/ingress.py --examples                        # Part 2 + Part 4.2
python hw10/validate_judge.py                            # Part 3.2-3.3
HF_HUB_OFFLINE=1 caffeinate -i python hw10/latency_experiment.py   # Part 4.3 (resumable)
streamlit run hw10/dashboard.py --server.port 8676      # Part 1
python tests/test_hw10.py && python scripts/verify_hw10.py
```

```
[PASS] file hw10/dashboard.py
[PASS] file hw10/evaluator.py
[PASS] file hw10/judge.py
[PASS] file hw10/ingress.py
[PASS] file hw10/validate_judge.py
[PASS] file hw10/latency_experiment.py
[PASS] file kafka_hw8/tracing.py
[PASS] file tests/test_hw10.py
[PASS] dashboard uses st.cache_data and st.cache_resource
[PASS] eval_cases.json has 20 drafts 20
[PASS] human_labels.csv has 20 labels in 1-5
[PASS] labels committed before judge results labels 2026-11-10T13:15:04-08:00 / judge
[PASS] spans have the required fields 540 spans
[PASS] trace example cache_hit present in spans 3ec85a84-1916-4794-9924-26c8ba6a8176
[PASS] trace example cache_miss present in spans fc5eeb2a-f33f-41e9-9723-8d64ea033229
[PASS] trace example failed present in spans 378f1087-ca1a-41a8-9847-a7ef7fb9e89c
[PASS] at least one ok end-to-end evaluation
[PASS] judge summary has Spearman, MAD, repeatability
[PASS] 50 confirmed cache misses 50

19/19 checks passed -> reports/hw10/verification.json
(data260) Pinal Pawar | s7875 data260-7875 %
```

SS-36: verify_hw10.py – self-check of all deliverables

AI Use

- **What the AI assistant (Claude) did:** read the HW10 assignment with me and explained the concepts (GEval/LLM-as-judge, Spearman rank correlation, repeatability, trace propagation and spans, p50/p95 , ingress, tracing hooks, evaluator, judge wrapper, validation and latency scripts, dashboard, tests, and drafted this report around my screenshots and results.
- **What I did:** ran everything on my MacBook Air (Kafka, Redis, MySQL, Ollama, the three agents), took every screenshot, and read the outputs. I **assigned all 20 human labels myself** with no model involved and committed them before the judge ran; I did not relabel after seeing the judge's scores.
- **What was wrong or needed fixing:** the database check showed the wrong function and table names (`engine`, `recall_notices` instead of `get_engine`, `notices`), and its first examples script did not print the Part 2 scores (a separate print command was added). Its judge rubric adds a safety sentence to every step, which Part 3 showed biases the judge toward the recall checklist; this is reported as a finding rather than fixed and re-run, because re-tuning on the same 20 frozen cases would invalidate the comparison.
- **Accuracy:** every number in this report comes from my runs (`reports/hw10/raw/`: spans.jsonl, requests.jsonl, eval_results.jsonl, part3_judge_runs.csv, part3_judge_summary.json, part4_latency_summary.json, trace_examples.json).

Spearman rank correlation measures whether two people **put things in the same order**, ignoring the exact scores.

How it works

1. Rank the 20 drafts by your labels, from best to worst.
2. Rank the same 20 drafts by the judge's scores.
3. Compare the two orderings.

The result is a number from **–1 to +1**:

Value	Meaning
+1	Same order: whatever you rated highest, the judge rated highest too
0	No relationship; the judge's order looks random compared with yours
–1	Opposite order: your best drafts are the judge's worst

Why ranks instead of raw scores? One judge can score everything a bit higher than you do, say always one point more, and still agree with you completely about which drafts are better. Spearman ignores that kind of offset and only asks whether the order matches. That's why the assignment pairs it with the **mean absolute difference**, which measures the size of the gap instead.

Ties: when several drafts share a score, they share an average rank. For example, if three drafts tie for 2nd, 3rd and 4th place, each gets rank 3. Your data had many ties (the judge gave 13 of 20 drafts a 4), and that makes Spearman less stable with only 20 cases.

Your result: –0.54. That's a moderately strong *opposite* ordering. The clearest examples:

- **c14 and c18:** you rated them **2** (among your lowest), and the judge rated them **5** (its